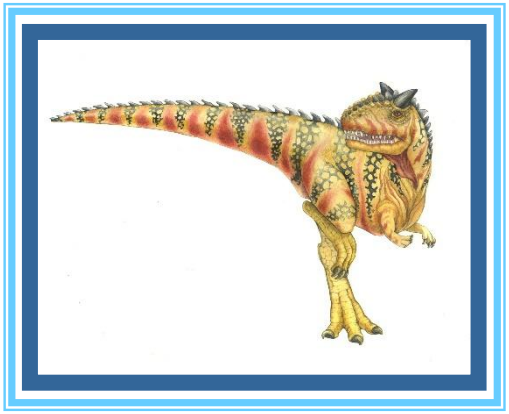
CS2303&2310: Operating Systems

School of Computer Science
Chentao Wu 吴晨涛
wuct@cs.sjtu.edu.cn

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

Chapter 7: Synchronization Examples



Chapter 7: Synchronization Examples

Explain the bounded-buffer, readers-writers, and dining philosophers synchronization problems.

Describe the tools used by Linux and Windows to solve synchronization problems.

Illustrate how POSIX and Java can be used to solve process synchronization problems.

Classical Problems of Synchronization

Classical problems used to test newly-proposed synchronization schemes

- Bounded-Buffer Problem
- Readers and Writers Problem
- Dining-Philosophers Problem

Bounded-Buffer Problem

n buffers, each can hold one item

Semaphore **`mutex`** initialized to the value 1

Semaphore **`full`** initialized to the value 0

Semaphore **`empty`** initialized to the value n

Bounded Buffer Problem (Cont.)

The structure of the producer process

```
while (true) {
        ...
        /* produce an item in next_produced */
        ...
    wait(empty);
    wait(mutex);
        ...
        /* add next produced to the buffer */
        ...
    signal(mutex);
    signal(full);
 }
```

Bounded Buffer Problem (Cont.)

The structure of the consumer process

```
  while (true) {
      wait(full);
      wait(mutex);
          ...
        /* remove an item from buffer to
next_consumed */
          ...
      signal(mutex);
      signal(empty);
          ...
        /* consume the item in next consumed */
          ...
     }
```

Readers-Writers Problem

A data set is shared among a number of concurrent processes

- **Readers** – only read the data set; they do ***not*** perform any updates
- **Writers** – can both read and write

Problem – allow multiple readers to read at the same time

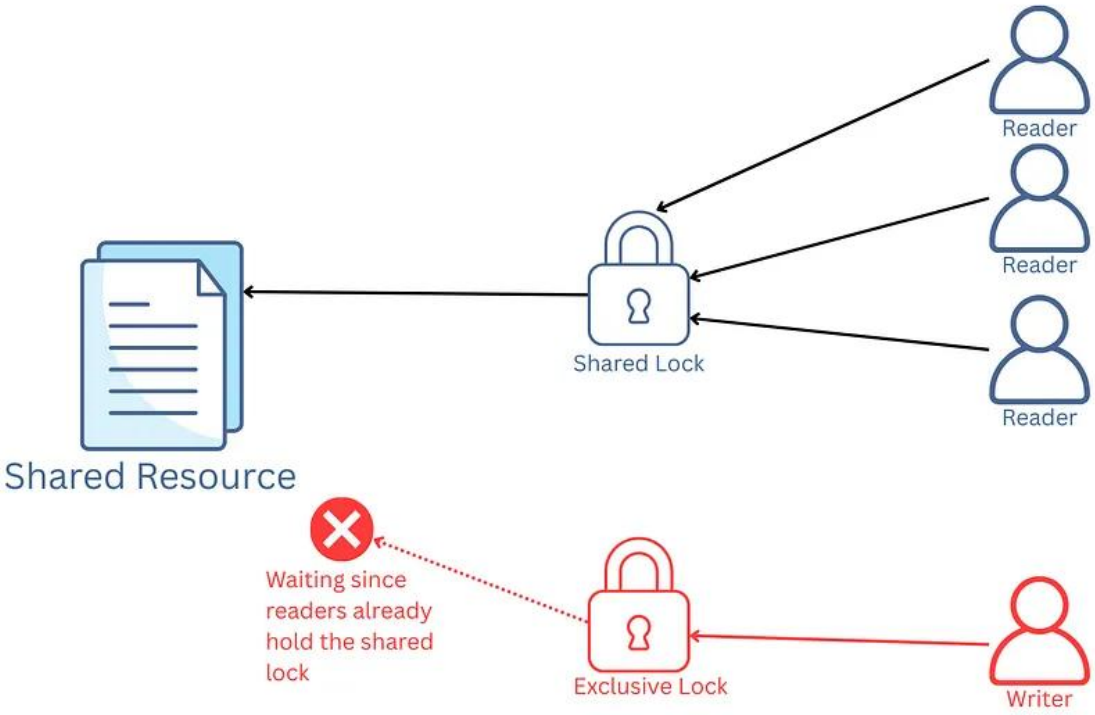
- Only one single writer can access the shared data at the same time

Several variations of how readers and writers are considered – all involve some form of priorities

Shared Data

- Data set
- Semaphore **`rw_mutex`** initialized to 1
- Semaphore **`mutex`** initialized to 1
- Integer **`read_count`** initialized to 0

Readers-Writers Lock



Readers-Writers Problem (Cont.)

The structure of a writer process

```
while (true) {
        wait(rw_mutex);
            ...
        /* writing is performed */
            ...
      signal(rw_mutex);
}
```

Readers-Writers Problem (Cont.)

The structure of a reader process

```
while (true){
            wait(mutex);
      read_count++;
      if (read_count == 1)
      wait(rw_mutex);
      signal(mutex);
         ...
            /* reading is performed */
         ...
      wait(mutex);
            read count--;
            if (read_count == 0)
            signal(rw_mutex);
      signal(mutex);
}
```

Readers-Writers Problem Variations

First variation – no reader kept waiting unless writer has permission to use shared object

Second variation – once writer is ready, it performs the write ASAP

Both may have starvation leading to even more variations

Problem is solved on some systems by kernel providing reader-writer locks

Read Copy Update (RCU)

RCU is a **lock-free** synchronization mechanism in Linux kernel

RCU stands for "**Read, Copy, Update**", which means that the updater thread first makes a copy of the data item, modifies the copy, and then replaces the original with the copy.

It works very well and has great scalability under read-heavy use cases.

Drawbacks of Reader-Writer lock (RWlock)

It is **expensive** and **complex** to implement, especially for fine-grained locking of individual data items.

It can cause **contention and scalability issues**, especially when the number of readers is large or the lock is held for a long time.

It can introduce **deadlocks and livelocks**, especially when the locking order is not well-defined or the lock is nested or reentrant.

Benefits of RCU

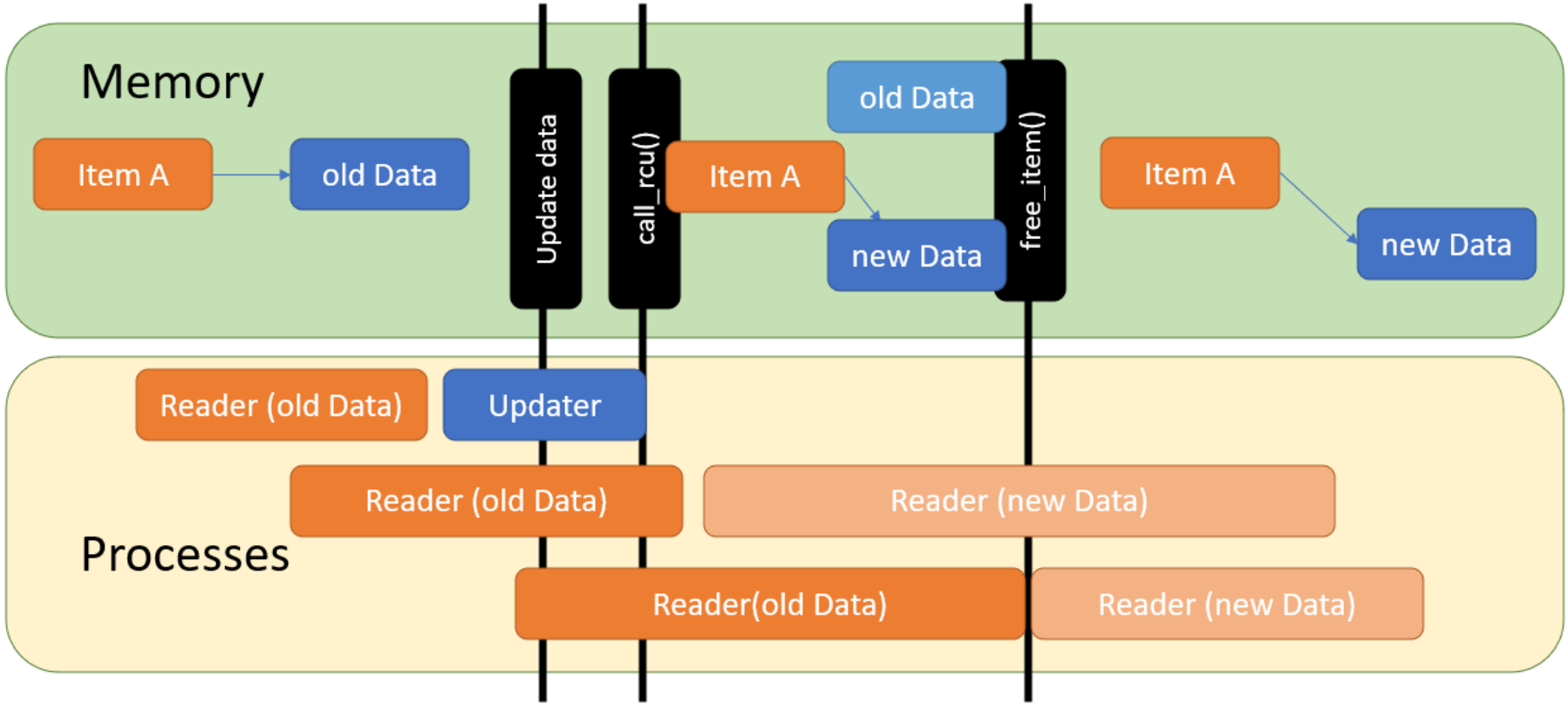
It is **faster and more scalable** for read-intensive workloads because the readers do not need to wait for the updaters or contend for shared resources.

It is **simpler and more reliable** for avoiding deadlocks because the readers do not need to acquire any locks or follow any ordering rules.

It is **more flexible and adaptable** for different scenarios because there are many variants of RCU that can handle different types of updates and readers.

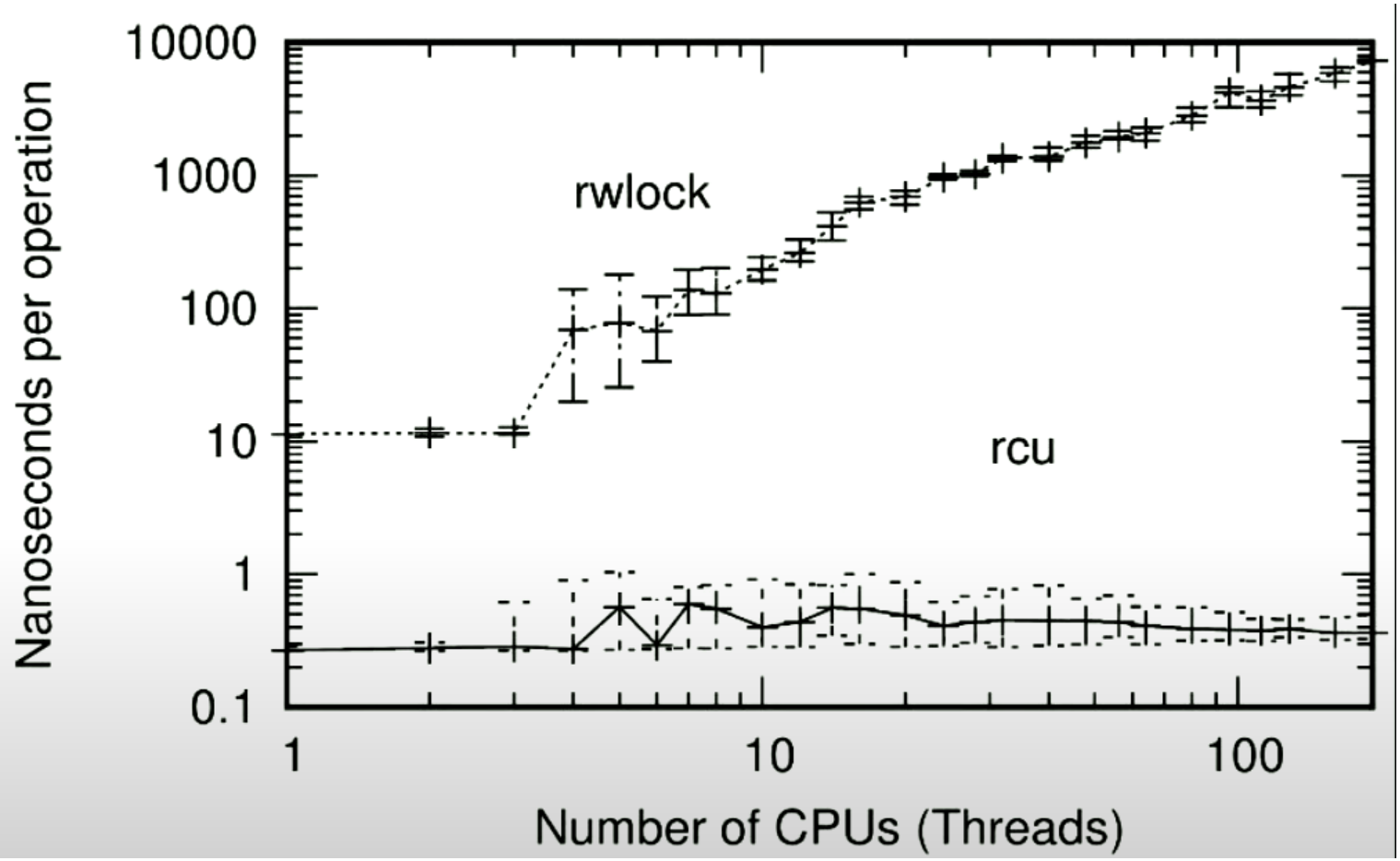
RCU

When the updater tries to update the data, the data doesn't get replaced instantly. The pointer will point to new data, but the old data still exists in the memory. This method prevents readers who are still accessing the old data from accessing the released memory and also allows updates before all readers leave the critical section.



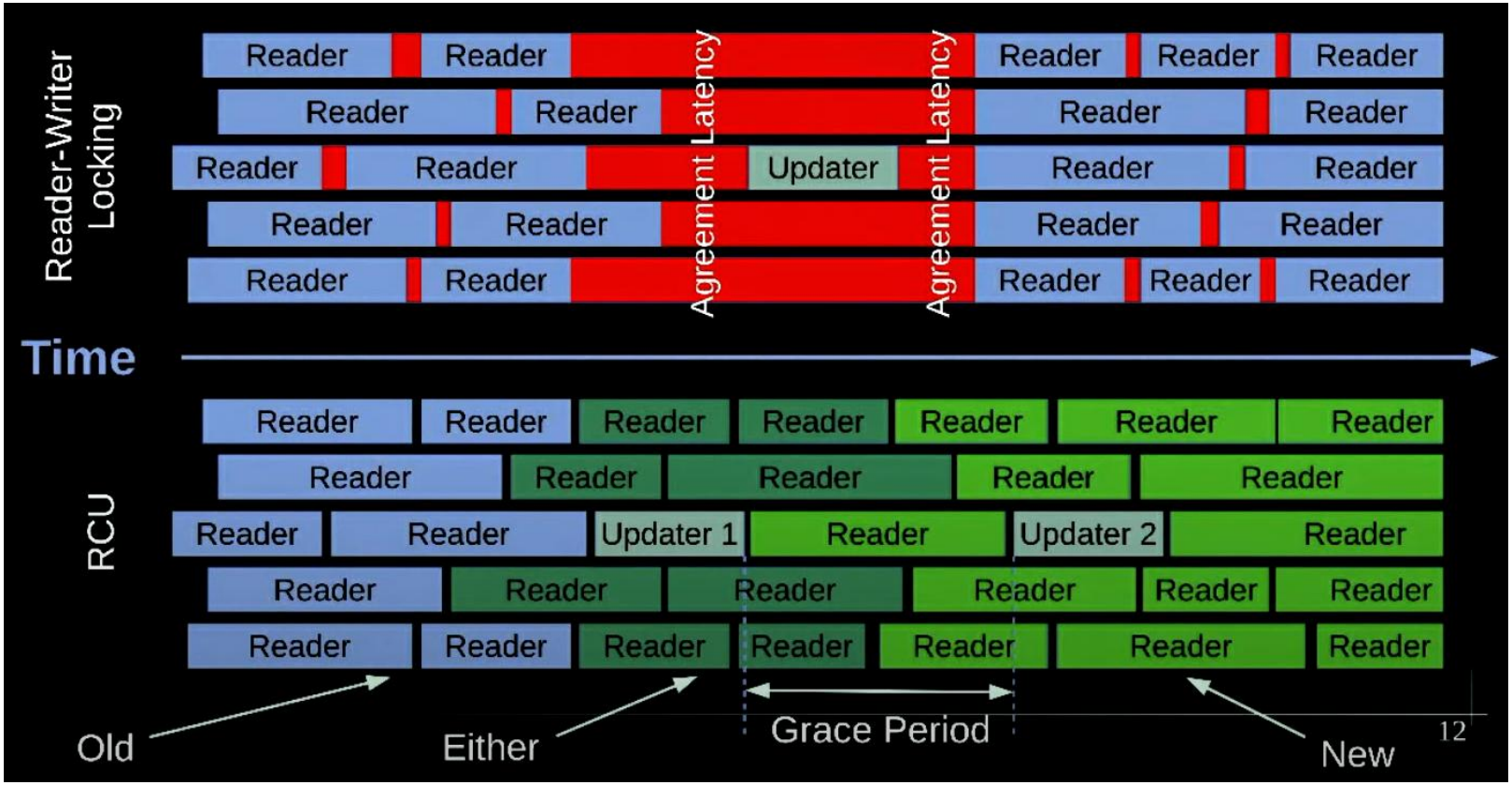
Comparison between RCU and RWlock

Scalability of RWlock and RCU

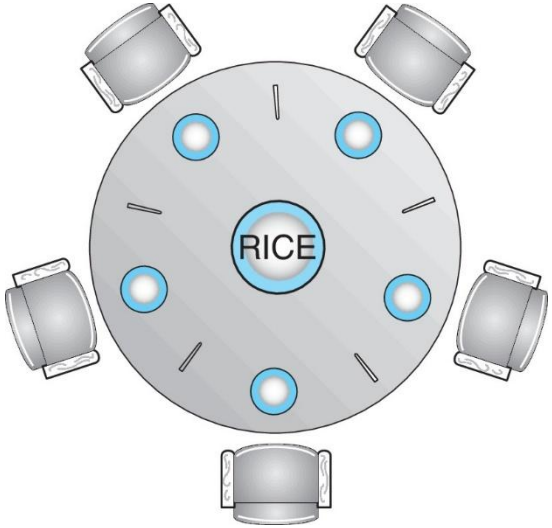


Comparison between RCU and RWlock

What it might look like in the CPUs while running these two methods


Reader-Writer Locking
Reader
Reader
Reader

Dining-Philosophers Problem



Philosophers spend their lives alternating thinking and eating

Don’t interact with their neighbors, occasionally try to pick up 2 chopsticks (one at a time) to eat from bowl

Need both to eat, then release both when done

In the case of 5 philosophers

Shared data

- Bowl of rice (data set)
- Semaphore chopstick [5] initialized to 1

Dining-Philosophers Problem Algorithm

Semaphore Solution

The structure of Philosopher *i*:

```
while (true){
     wait (chopstick[i] );
    wait (chopStick[ (i + 1) % 5] );

     /* eat for awhile */

    signal (chopstick[i] );
    signal (chopstick[ (i + 1) % 5] );

     /* think for awhile */

}
```

What is the problem with this algorithm?

Monitor Solution to Dining Philosophers

```
monitor DiningPhilosophers
{
   enum { THINKING; HUNGRY, EATING) state [5] ;
   condition self [5];

   void pickup (int i) {
          state[i] = HUNGRY;
          test(i);
          if (state[i] != EATING) self[i].wait;
   }

    void putdown (int i) {
          state[i] = THINKING;
                   // test left and right neighbors
           test((i + 4) % 5);
           test((i + 1) % 5);
   }
```

Solution to Dining Philosophers (Cont.)

```
   void test (int i) {
           if ((state[(i + 4) % 5] != EATING) &&
           (state[i] == HUNGRY) &&
           (state[(i + 1) % 5] != EATING) ) {
                state[i] = EATING ;
          self[i].signal () ;
           }
    }

        initialization_code() {
          for (int i = 0; i < 5; i++)
          state[i] = THINKING;
        }
}
```

Solution to Dining Philosophers (Cont.)

Each philosopher *i* invokes the operations **`pickup()`** and **`putdown()`** in the following sequence:

```
DiningPhilosophers.pickup(i);

    /** EAT **/

DiningPhilosophers.putdown(i);
```

No deadlock, but starvation is possible

Kernel Synchronization - Windows

Uses interrupt masks to protect access to global resources on uniprocessor systems

Uses **spinlocks** on multiprocessor systems

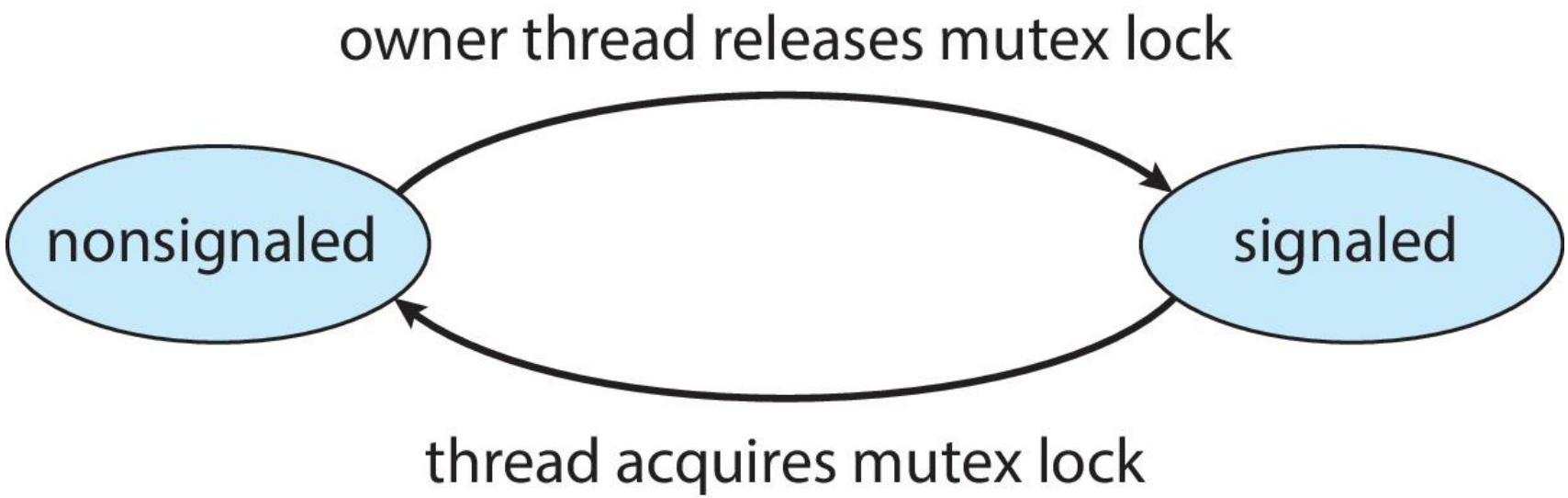
- Spinlocking-thread will never be preempted

Also provides **dispatcher objects** user-land which may act mutexes, semaphores, events, and timers

- **Events**
  - An event acts much like a condition variable
- Timers notify one or more thread when time expired
- Dispatcher objects either **signaled-state** (object available) or **non-signaled state** (thread will block)

Kernel Synchronization - Windows

Mutex dispatcher object



Linux Synchronization

Linux:

- Prior to kernel Version 2.6, disables interrupts to implement short critical sections
- Version 2.6 and later, fully preemptive

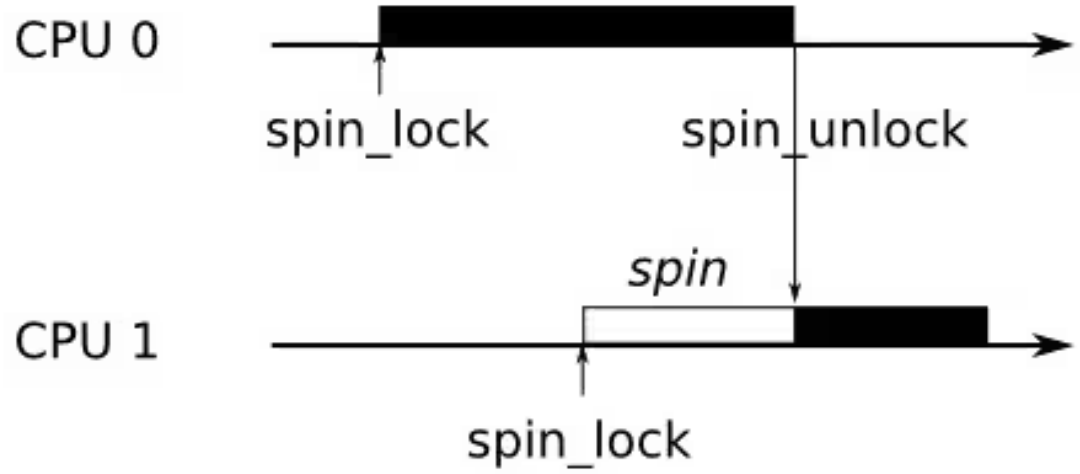
Linux provides:

- Semaphores
- atomic integers
- spinlocks
- reader-writer versions of both (e.g., **RCU**)

On single-CPU system, spinlocks replaced by enabling and disabling kernel preemption

Spinlock

Spinlock typically with Compare-and-Swap instructions



Linux Synchronization

Atomic variables

`atomic_t` is the type for atomic integer

Consider the variables

```
atomic_t counter;
int value;
```

Atomic Operation	*Effect*
`atomic_set(&counter,5);`	`counter = 5`
`atomic_add(10,&counter);`	`counter = counter + 10`
`atomic_sub(4,&counter);`	`counter = counter - 4`
`atomic_inc(&counter);`	`counter = counter + 1`
`value = atomic_read(&counter);`	`value = 12`

POSIX Synchronization

POSIX API provides

- mutex locks
- semaphores
- condition variable

Widely used on UNIX, Linux, and macOS

POSIX Mutex Locks

Creating and initializing the lock

```
#include <pthread.h>

pthread_mutex_t mutex;

/* create and initialize the mutex lock */
pthread_mutex_init(&mutex,NULL);
```

Acquiring and releasing the lock

```
/* acquire the mutex lock */
pthread_mutex_lock(&mutex);

/* critical section */

/* release the mutex lock */
pthread_mutex_unlock(&mutex);
```

POSIX Semaphores

POSIX provides two versions – **named** and **unnamed**.

Named semaphores can be used by unrelated processes, unnamed cannot.

POSIX Named Semaphores

Creating an initializing the semaphore:

```
#include <semaphore.h>
sem_t *sem;

/* Create the semaphore and initialize it to 1 */
sem = sem_open("SEM", O_CREAT, 0666, 1);
```

Another process can access the semaphore by referring to its name **SEM**.

Acquiring and releasing the semaphore:

```
/* acquire the semaphore */
sem_wait(sem);

/* critical section */

/* release the semaphore */
sem_post(sem);
```

POSIX Unnamed Semaphores

Creating an initializing the semaphore:

```
#include <semaphore.h>
sem_t sem;

/* Create the semaphore and initialize it to 1 */
sem_init(&sem, 0, 1);
```

Acquiring and releasing the semaphore:

```
/* acquire the semaphore */
sem_wait(&sem);

/* critical section */

/* release the semaphore */
sem_post(&sem);
```

POSIX Condition Variables

Since POSIX is typically used in C/C++ and these languages do not provide a monitor, POSIX condition variables are associated with a POSIX mutex lock to provide mutual exclusion: Creating and initializing the condition variable:

```
pthread_mutex_t mutex;
pthread_cond_t cond_var;

pthread_mutex_init(&mutex,NULL);
pthread_cond_init(&cond_var,NULL);
```

POSIX Condition Variables

Thread waiting for the condition **`a == b`** to become true:

```
pthread_mutex_lock(&mutex);
while (a != b)
      pthread_cond_wait(&cond_var, &mutex);

pthread_mutex_unlock(&mutex);
```

Thread signaling another thread waiting on the condition variable:

```
pthread_mutex_lock(&mutex);
a = b;
pthread_cond_signal(&cond_var);
pthread_mutex_unlock(&mutex);
```

Java Synchronization

Java provides rich set of synchronization features:

- Java monitors
- Reentrant locks
- Semaphores
- Condition variables

Java Monitors

Every Java object has associated with it a single lock.

If a method is declared as **`synchronized`**, a calling thread must own the lock for the object.

If the lock is owned by another thread, the calling thread must wait for the lock until it is released.

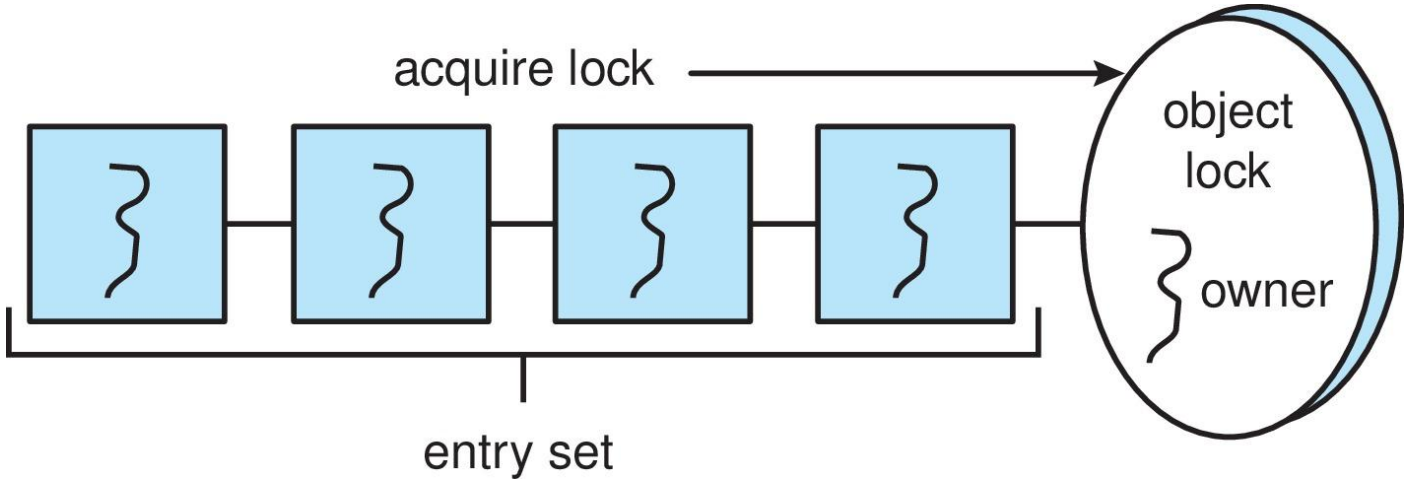
Locks are released when the owning thread exits the **`synchronized`** method.

Bounded Buffer – Java Synchronization

```
public class BoundedBuffer<E>
{
   private static final int BUFFER_SIZE = 5;

   private int count, in, out;
   private E[] buffer;

   public BoundedBuffer() {
      count = 0;
      in = 0;
      out = 0;
      buffer = (E[]) new Object[BUFFER_SIZE];
   }

   /* Producers call this method */
   public synchronized void insert(E item) {
      /* See Figure 7.11 */
   }

   /* Consumers call this method */
   public synchronized E remove() {
      /* See Figure 7.11 */
   }
}
```

Java Synchronization

A thread that tries to acquire an unavailable lock is placed in the object's **entry set**:

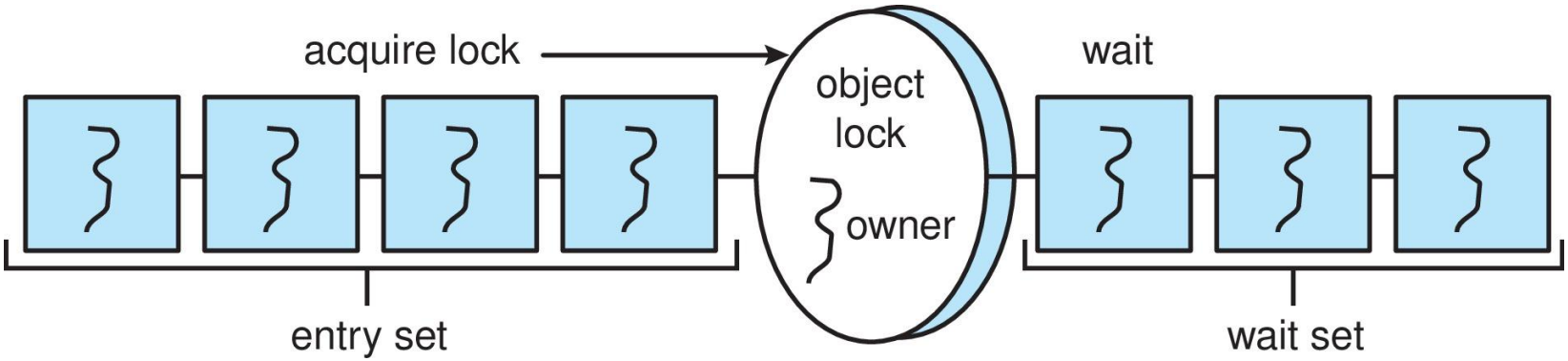


Java Synchronization

Similarly, each object also has a **wait set**.

When a thread calls **`wait()`**:

1. It releases the lock for the object
2. The state of the thread is set to blocked
3. The thread is placed in the wait set for the object



Java Synchronization

A thread typically calls wait() when it is waiting for a condition to become true.

How does a thread get notified?

When a thread calls **`notify()`**:

1. An arbitrary thread T is selected from the wait set
2. T is moved from the wait set to the entry set
3. Set the state of T from blocked to runnable.

T can now compete for the lock to check if the condition it was waiting for is now true.

Bounded Buffer – Java Synchronization

```
/* Producers call this method */
public synchronized void insert(E item) {
   while (count == BUFFER_SIZE) {
      try {
         wait();
      }
      catch (InterruptedException ie) { }
   }

   buffer[in] = item;
   in = (in + 1) % BUFFER_SIZE;
   count++;

   notify();
}
```

Bounded Buffer – Java Synchronization

```
/* Consumers call this method */
public synchronized E remove() {
   E item;

   while (count == 0) {
      try {
         wait();
      }
      catch (InterruptedException ie) { }
   }

   item = buffer[out];
   out = (out + 1) % BUFFER_SIZE;
   count--;

   notify();

   return item;
}
```

Java Reentrant Locks

Similar to mutex locks

The **`finally`** clause ensures the lock will be released in case an exception occurs in the **`try`** block.

```
Lock key = new ReentrantLock();

key.lock();
try {
   /* critical section */
}
finally {
   key.unlock();
}
```

Java Semaphores

Constructor:

```
Semaphore(int value);
```

Usage:

```
Semaphore sem = new Semaphore(1);

try {
   sem.acquire();
   /* critical section */
}
catch (InterruptedException ie) { }
finally {
   sem.release();
}
```

Java Condition Variables

Condition variables are associated with an **`ReentrantLock`**.

Creating a condition variable using **`newCondition()`** method of **`ReentrantLock`**:

```
Lock key = new ReentrantLock();
Condition condVar = key.newCondition();
```

A thread waits by calling the **`await()`** method, and signals by calling the **`signal()`** method.

Java Condition Variables

Example:

Five threads numbered 0 .. 4

Shared variable **`turn`** indicating which thread's turn it is.

Thread calls **`doWork()`** when it wishes to do some work. (But it may only do work if it is their turn.

If not their turn, wait

If their turn, do some work for awhile ……

When completed, notify the thread whose turn is next.

Necessary data structures:

```
Lock lock = new ReentrantLock();
Condition[] condVars = new Condition[5];

for (int i = 0; i < 5; i++)
   condVars[i] = lock.newCondition();
```

Java Condition Variables

```
/* threadNumber is the thread that wishes to do some work */
public void doWork(int threadNumber)
{
   lock.lock();

   try {
      /**
       * If it's not my turn, then wait
       * until I'm signaled.
       */
      if (threadNumber != turn)
         condVars[threadNumber].await();

      /**
       * Do some work for awhile ...
       */

      /**
       * Now signal to the next thread.
       */
      turn = (turn + 1) % 5;
      condVars[turn].signal();
   }
   catch (InterruptedException ie) { }
   finally {
      lock.unlock();
   }
}
```

Alternative Approaches

Transactional Memory

OpenMP

Functional Programming Languages

Transactional Memory

Consider a function update() that must be called atomically. One option is to use mutex locks:

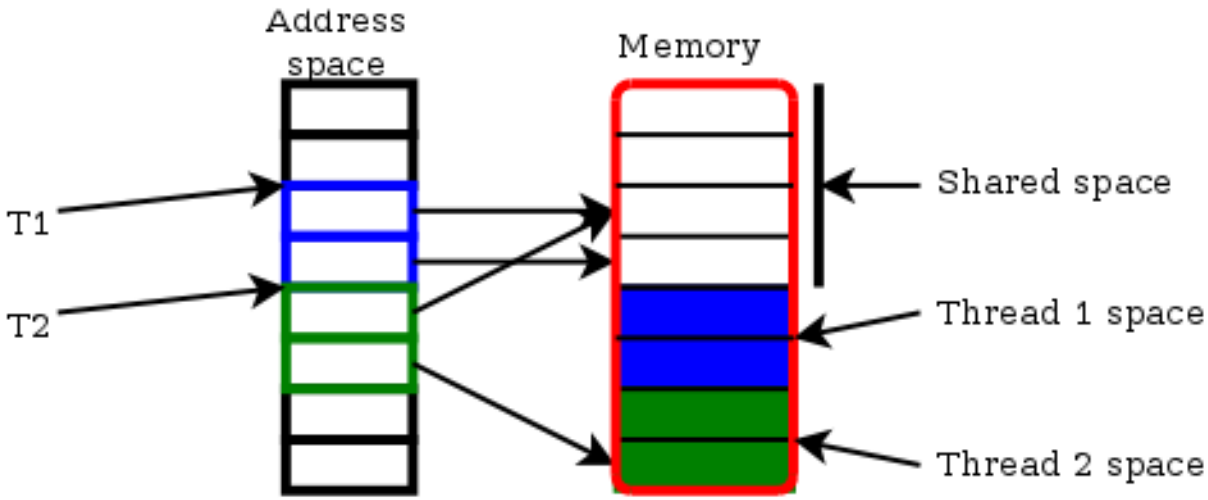
```
void update ()
{
   acquire();

   /* modify shared data */

   release();
}
```

A **memory transaction** is a sequence of read-write operations to memory that are performed atomically. A transaction can be completed by adding **`atomic{S}`** which ensure statements in **`S`** are executed atomically:

```
void update ()
{
   atomic {
      /* modify shared data */
   }
}
```

Transactional Memory

The diagram at left shows two threads (T1, T2) running, each with their address space pointing in to the shared region.



OpenMP

OpenMP is a set of compiler directives and API that support parallel progamming.

```
void update(int value)
{
    #pragma omp critical
    {
        count += value
    }
}
```

The code contained within the **`#pragma omp critical`** directive is treated as a critical section and performed atomically.

Functional Programming Languages

Functional programming languages offer a different paradigm than procedural languages in that they do not maintain state.

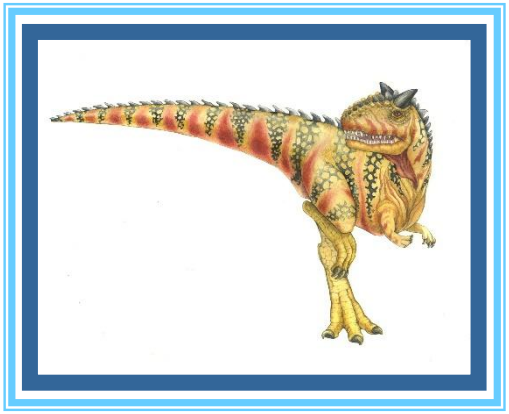
Variables are treated as immutable and cannot change state once they have been assigned a value.

There is increasing interest in functional languages such as Erlang and Scala for their approach in handling data races.

Homework

- Exercises at the end of Chapter 7 (OS book, 10th Edition)
  - 7.8, 7.11, 7.16

Data Consistency



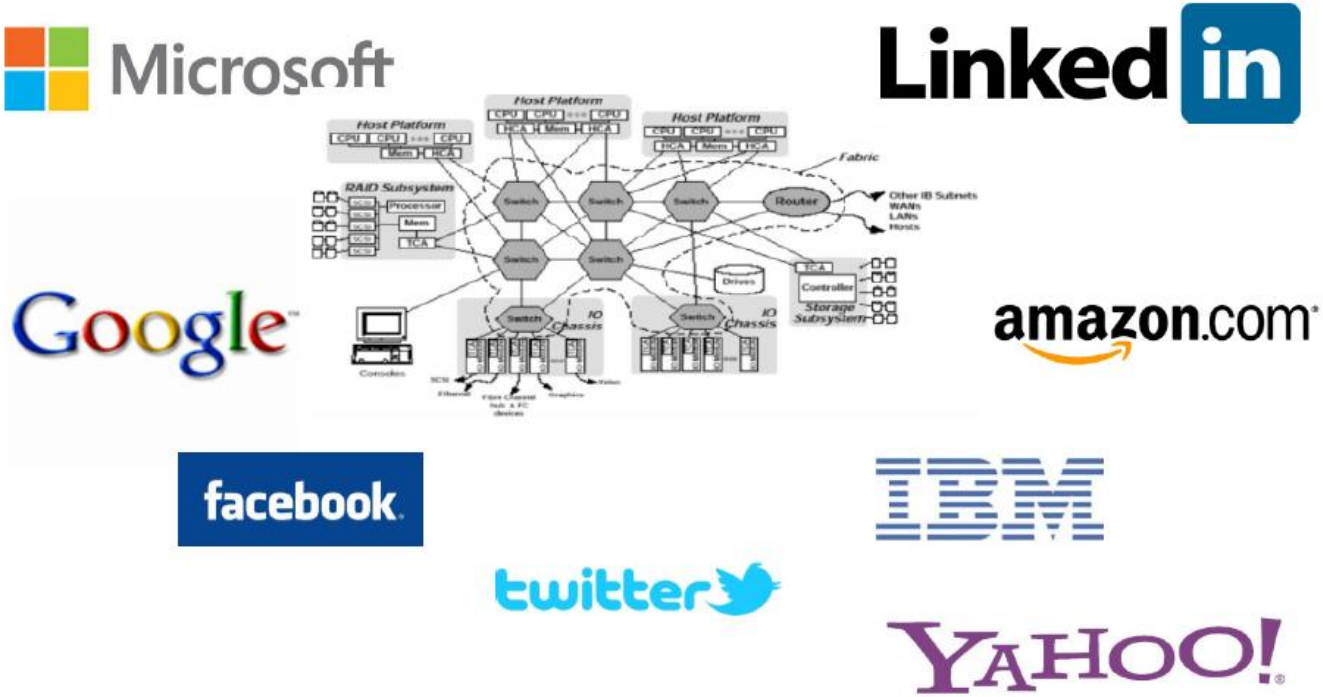
Data Consistency

Data Consistency:

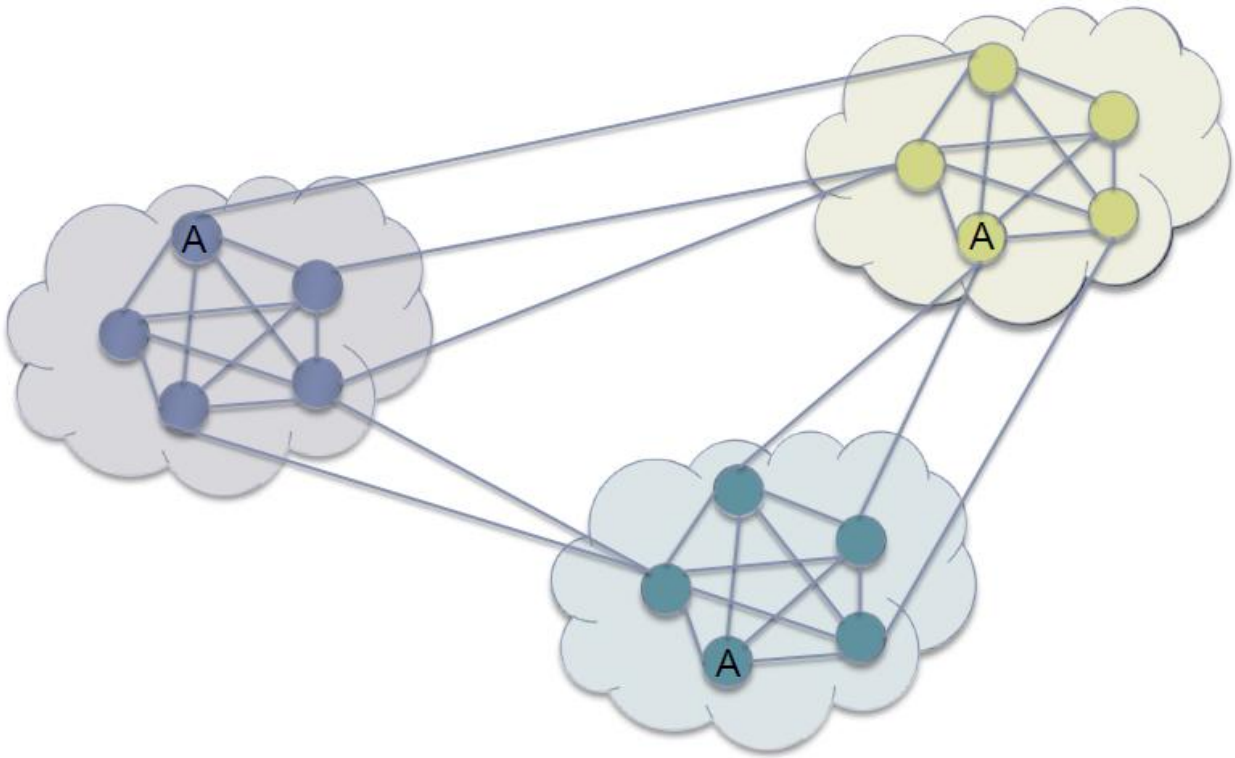
Data consistency refers to when same data kept at different places do not match.

Source: wikipedia

Today's Data Share Systems (1)



Today's Data Share Systems (2)



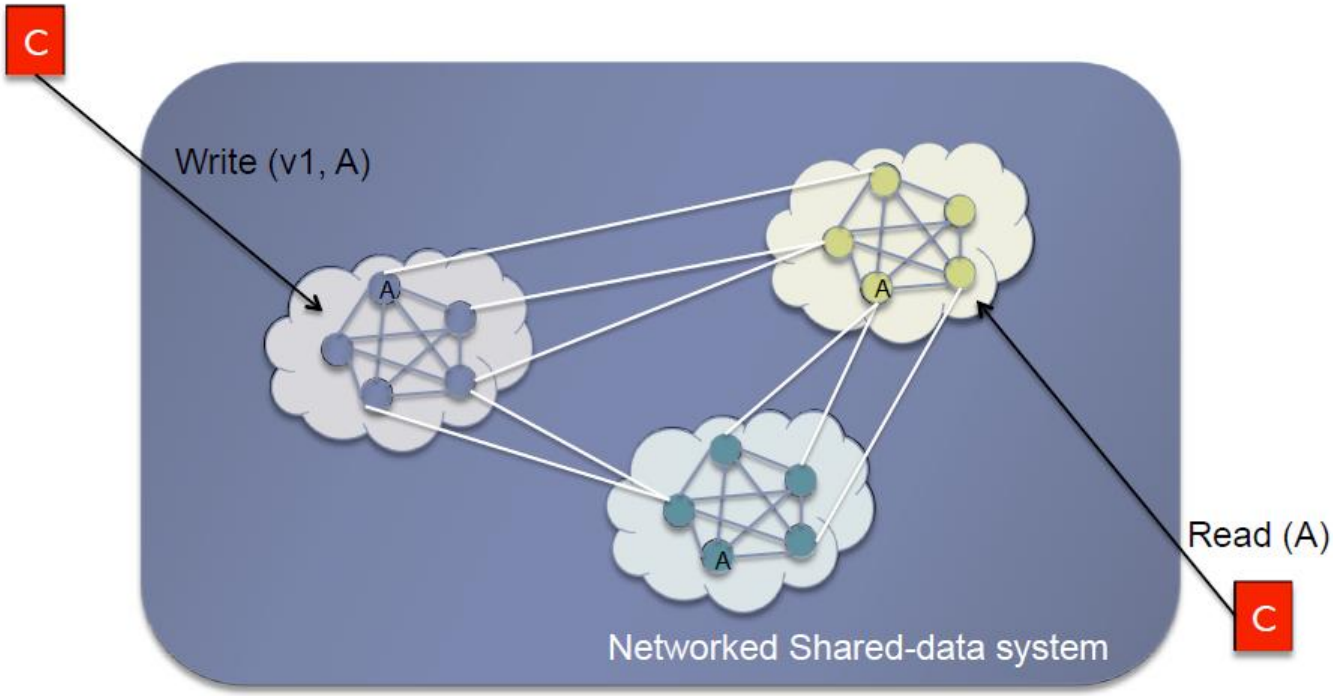
Fundamental Properties

- **Consistency**
  - (informally) “every request receives the right response”
  - E.g. If I get my shopping list on Amazon I expect it contains all the previously selected items
- **Availability**
  - (informally) “each request eventually receives a response”
  - E.g. eventually I access my shopping list
- **tolerance to network Partitions**
  - (informally) “servers can be partitioned in to multiple groups that cannot communicate with one other”

The CAP Theorem

- The CAP Theorem (Eric Brewer):
- **One can achieve at most two of the following:**
  - **Data Consistency**
  - **System Availability**
  - **Tolerance to network Partitions**
- Was first made as a conjecture At *PODC 2000* by Eric Brewer
- The Conjecture was formalized and confirmed by MIT researchers Seth Gilbert and Nancy Lynch in 2002

Proof



CAP

consistency

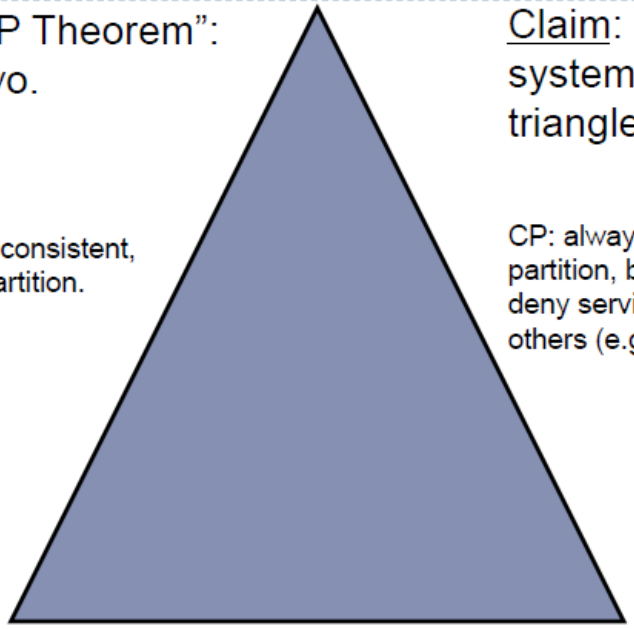
C

Fox&Brewer "CAP Theorem":
C-A-P: choose two.

Claim: every distributed system is on one side of the triangle.

CA: available, and consistent, unless there is a partition.

CP: always consistent, even in a partition, but a reachable replica may deny service without agreement of the others (e.g., quorum).



A

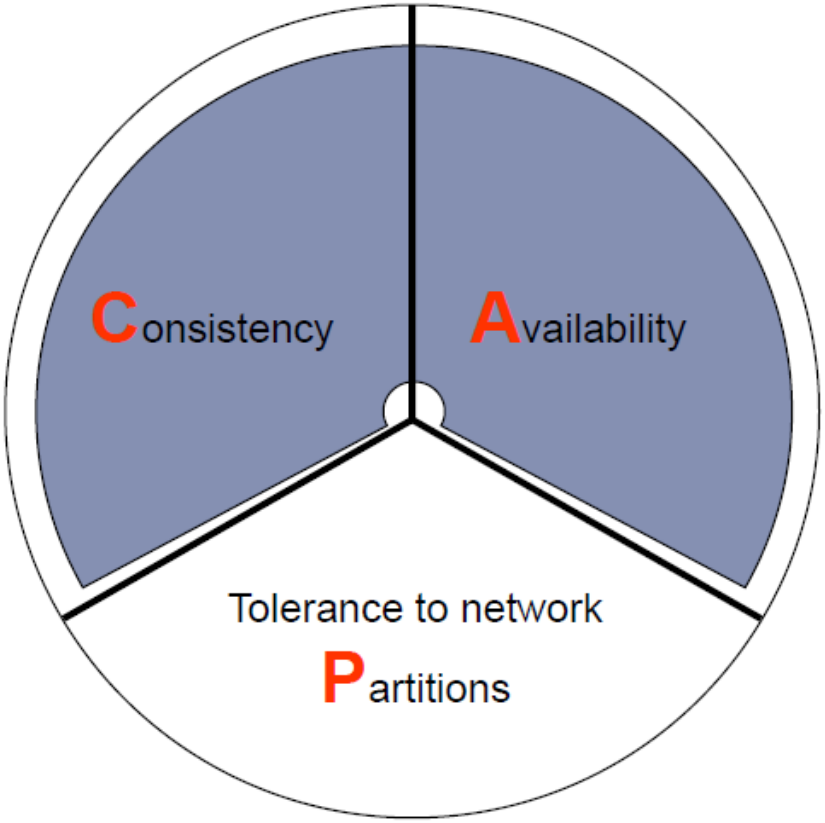
Availability

AP: a reachable replica provides service even in a partition, but may be inconsistent.

P

Partition-resilience

Forfeit Partitions



Examples

- Single-site databases
- Cluster databases
- LDAP
- Fiefdoms

Traits

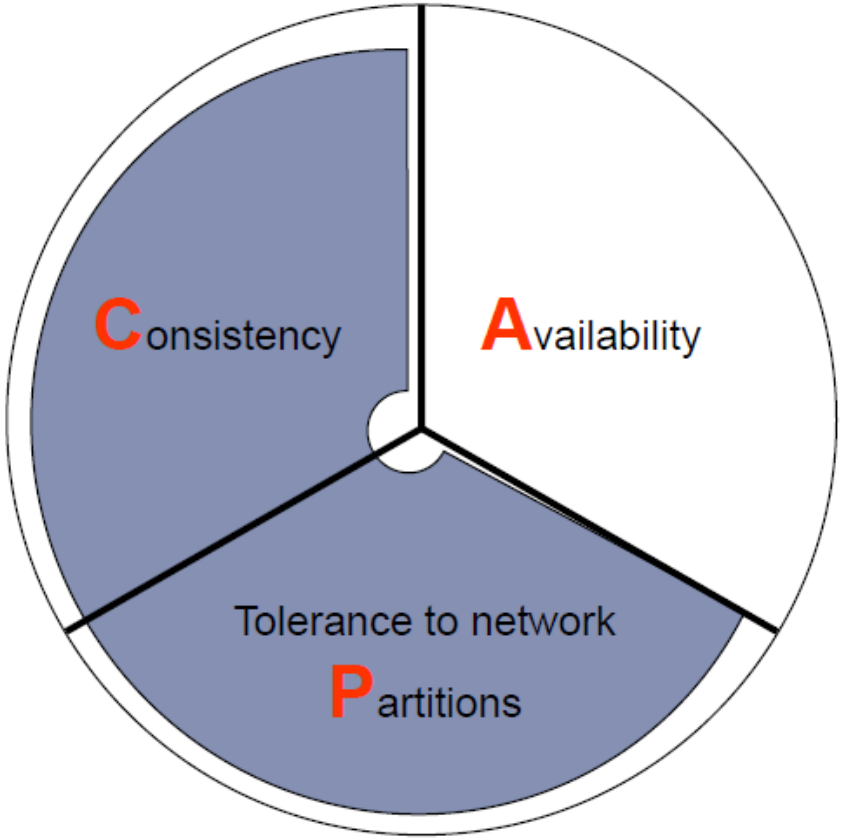
- 2-phase commit
- cache validation protocols
- The "inside"

Observations

- CAP states that in case of failures you can have at most two of these three properties for any shared-data system

- To scale out, you have to distribute resources.
  - P in not really an option but rather a need
  - The real selection is among consistency or availability
  - In almost all cases, you would choose availability over consistency

Forfeit Availability



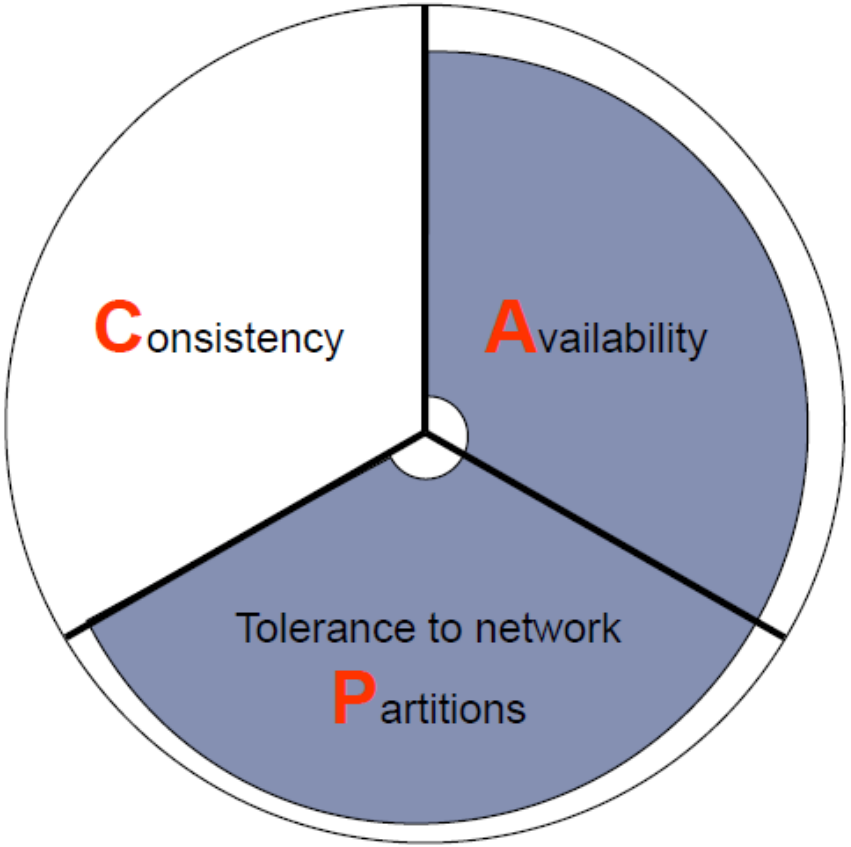
Examples

- Distributed databases
- Distributed locking
- Majority protocols

Traits

- Pessimistic locking
- Make minority partitions unavailable

Forfeit Consistency



Examples

- Coda
- Web caching
- DNS
- Emissaries

Traits

- expirations/leases
- conflict resolution
- Optimistic
- The "outside"

Consistency Boundary Summary

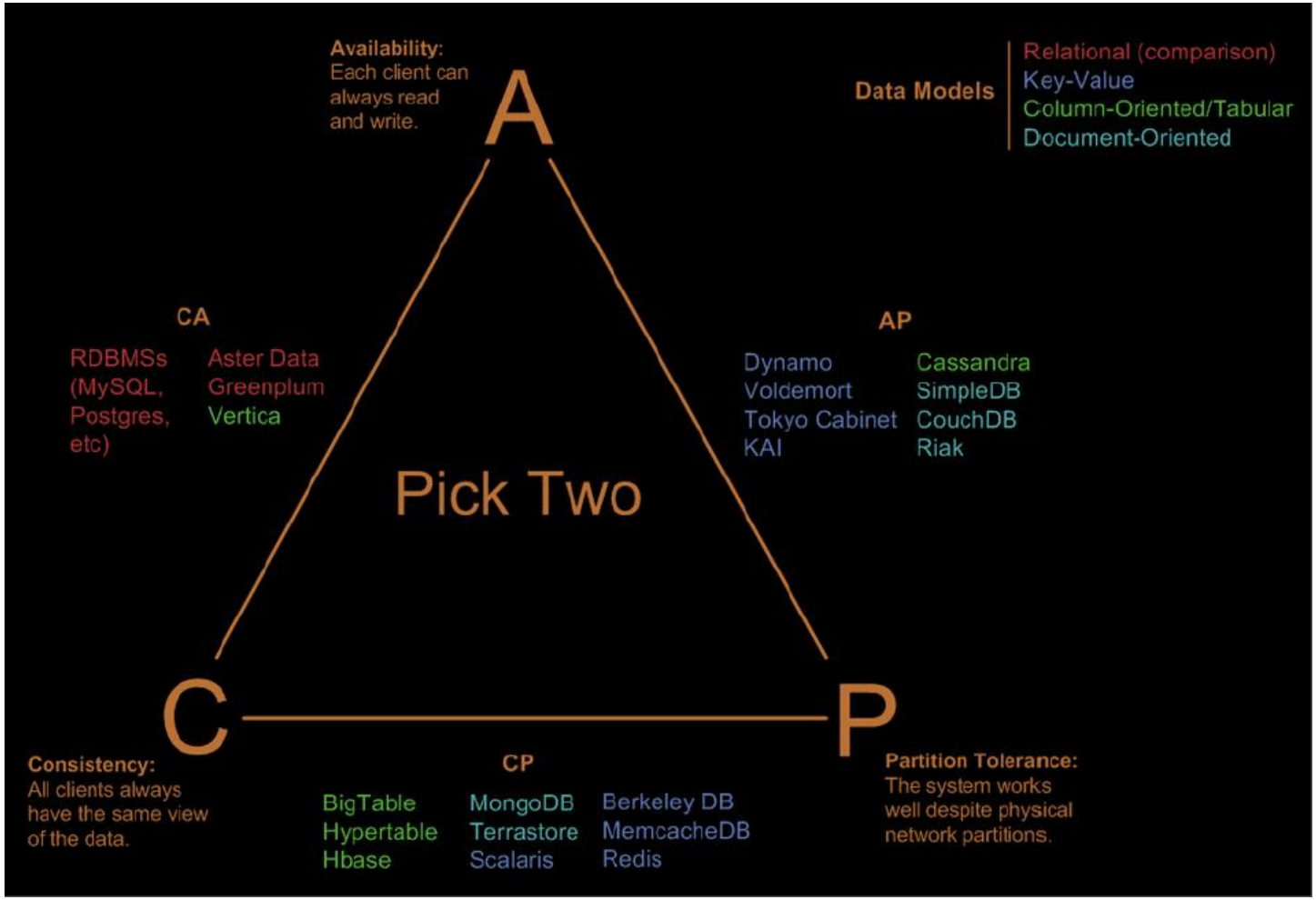
- We can have consistency & availability within a cluster.
- No partitions within boundary!

- **OS/Networking better at A than C**

- **Database/Storage better at C than A**

- Wide-area databases can’t have both
- Disconnected clients can’t have both

CAP in Database System



Another CAP - BASE

BASE stands for Basically Available Soft State Eventually Consistent system.

Basically Available: the system available most of the time and there could exists a subsystems temporarily unavailable

Soft State: data are “volatile” in the sense that their persistence is in the hand of the user that must take care of refresh them

Eventually Consistent: the system eventually converge to a consistent state

Another CAP - ACID

Relation among ACID and CAP is core complex

Atomicity: every operation is executed in “all-or-nothing” fashion

Consistency: every transaction preserves the consistency constraints on data

Integrity: transaction does not interfere. Every transaction is executed as it is the only one in the system

Durability: after a commit, the updates made are permanent regardless possible failures

CAP vs. ACID

CAP

C here looks to single-copy consistency

A here look to the service/data availability

ACID

C here looks to constraints on data and data model

A looks to atomicity of operation and it is always ensured

I is deeply related to CAP. I can be ensured in at most one partition

D is independent from CAP

2 of 3 is misleading (1)

In principle every system should be designed to ensure **both C and A in normal situation**

When a partition occurs the decision **among C and A can be taken**

When **the partition is resolved** the system takes corrective action coming back to work in normal situation

2 of 3 is misleading (2)

Partitions are rare events

there are little reasons to forfeit by design C or A

Systems evolve along time

Depending on the specific partition, service or data, the decision about the property to be sacrificed can change

C, A and P are measured according to continuum

Several level of Consistency (e.g. ACID vs BASE)

Several level of Availability

Several degree of partition severity

Consistency/Latency Tradeoff (1)

CAP does not force designers to give up A or C but why there exists a lot of systems trading C?

CAP does not explicitly talk about latency…

… however latency is crucial to get the essence of CAP

Consistency/Latency Tradeoff (2)

High Availability

- High Availability is a strong requirement of modern shared-data systems

Replication

- To achieve High Availability, data and services must be replicated

Consistency

- Replication impose consistency maintenance

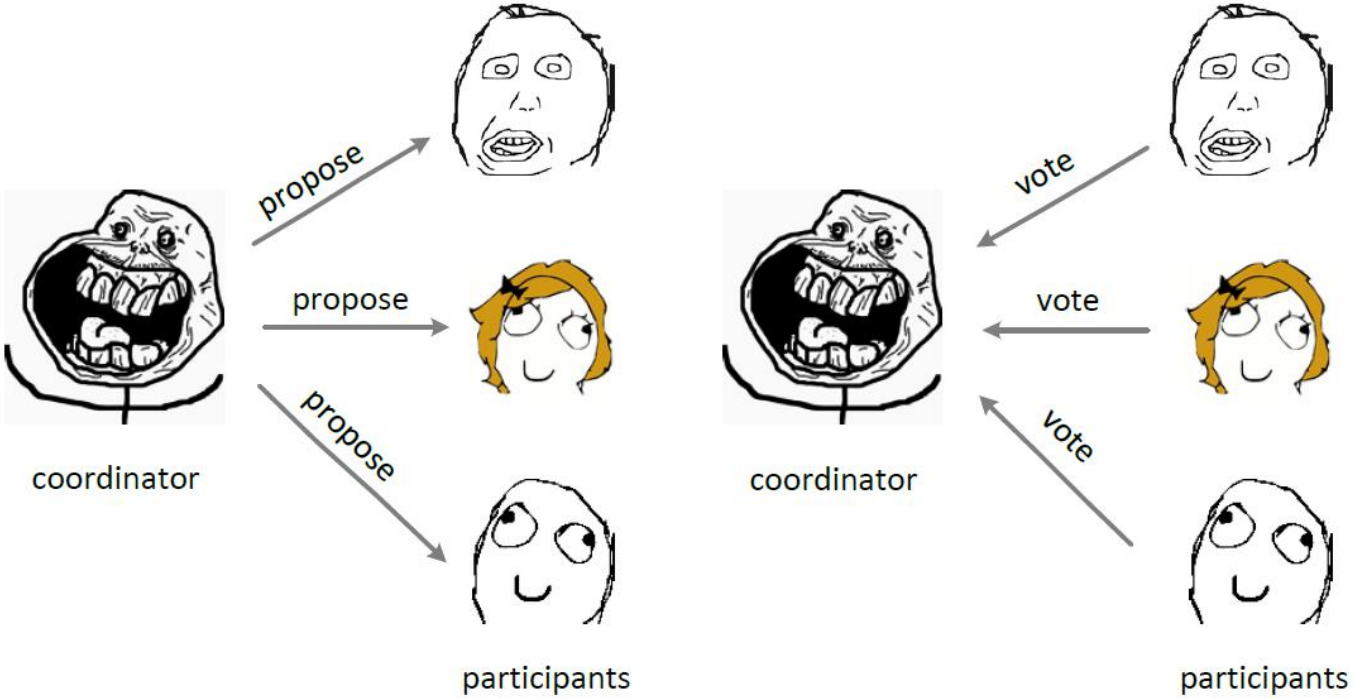
Latency

- Every form of consistency requires communication and a stronger consistency requires higher latency

2PC: Two Phase Commit Protocol

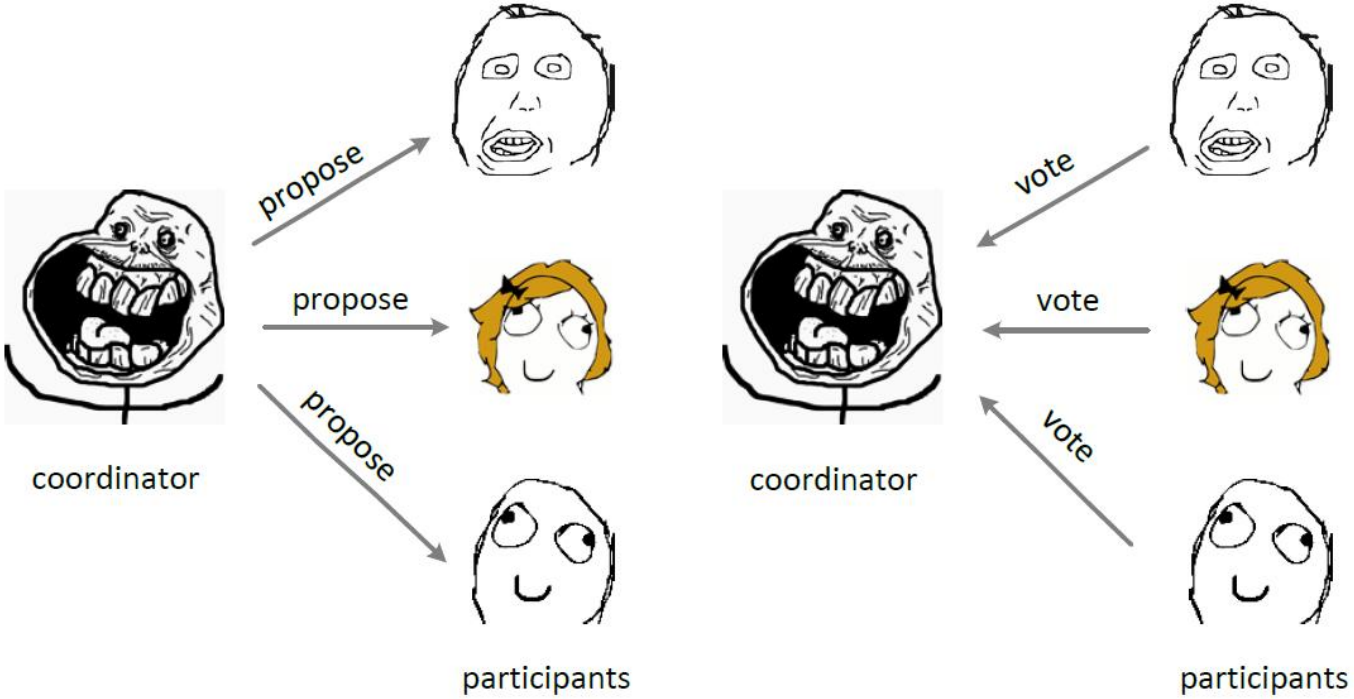
Coordinator: propose a vote to other nodes

Participants/Cohorts: send a vote to coordinator



2PC: Two Phase Commit Protocol

Phase 1: Coordinator propose a vote, and wait for the response of participants

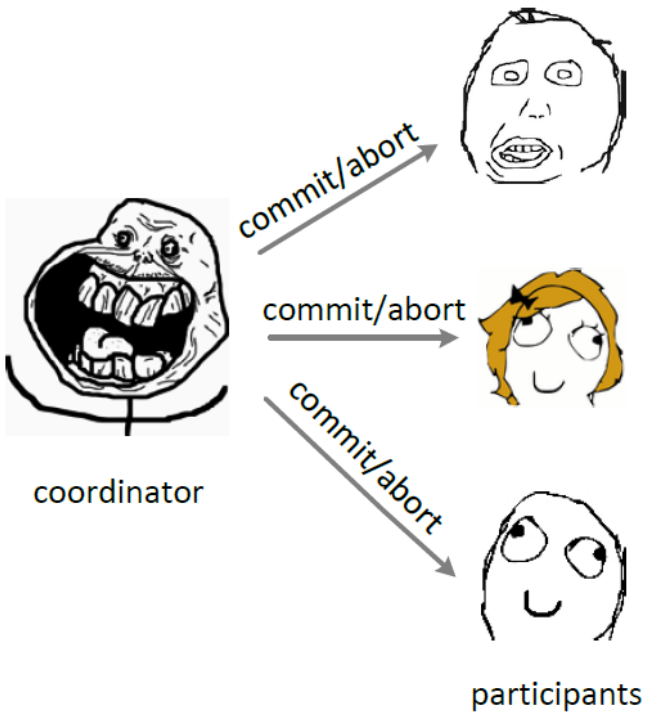


2PC: Two Phase Commit Protocol

Phase 2: Coordinator commits or aborts the transaction according to the participants' feedback

If all agree, commit

If any one disagree, abort



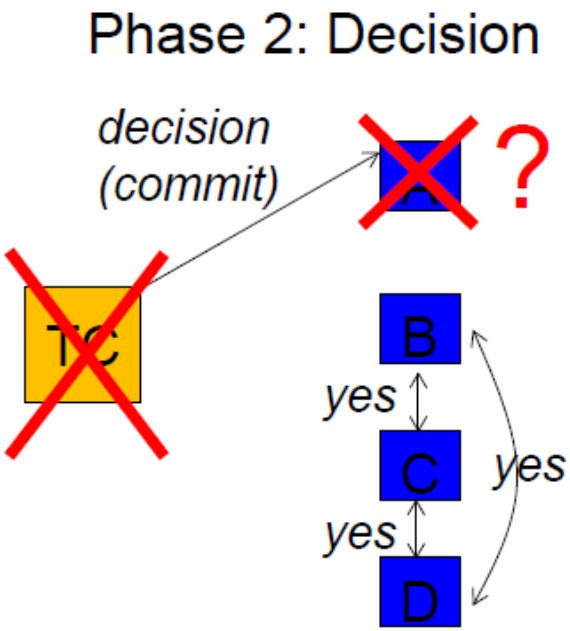
Problem of 2PC

Scenario:

- TC sends commit decision to A, A gets it and commits, and then both TC and A crash
- B, C, D, who voted Yes, now need to wait for TC or A to reappear (w/ mutexes locked)

They can't commit or abort, as they don't know what A responded

- If that takes a long time (e.g., a human must replace hardware), then availability suffers
- If TC is also participant, as it typically is, then this protocol is vulnerable to a single-node failure (the TC's failure)!



This is why 2 phase commit is called a **blocking protocol**

In context of consensus requirements: 2PC is **safe**, but not **live**

3PC: Three Phase Commit Protocol (1)

Goal: Turn 2PC into **a live (non-blocking) protocol**

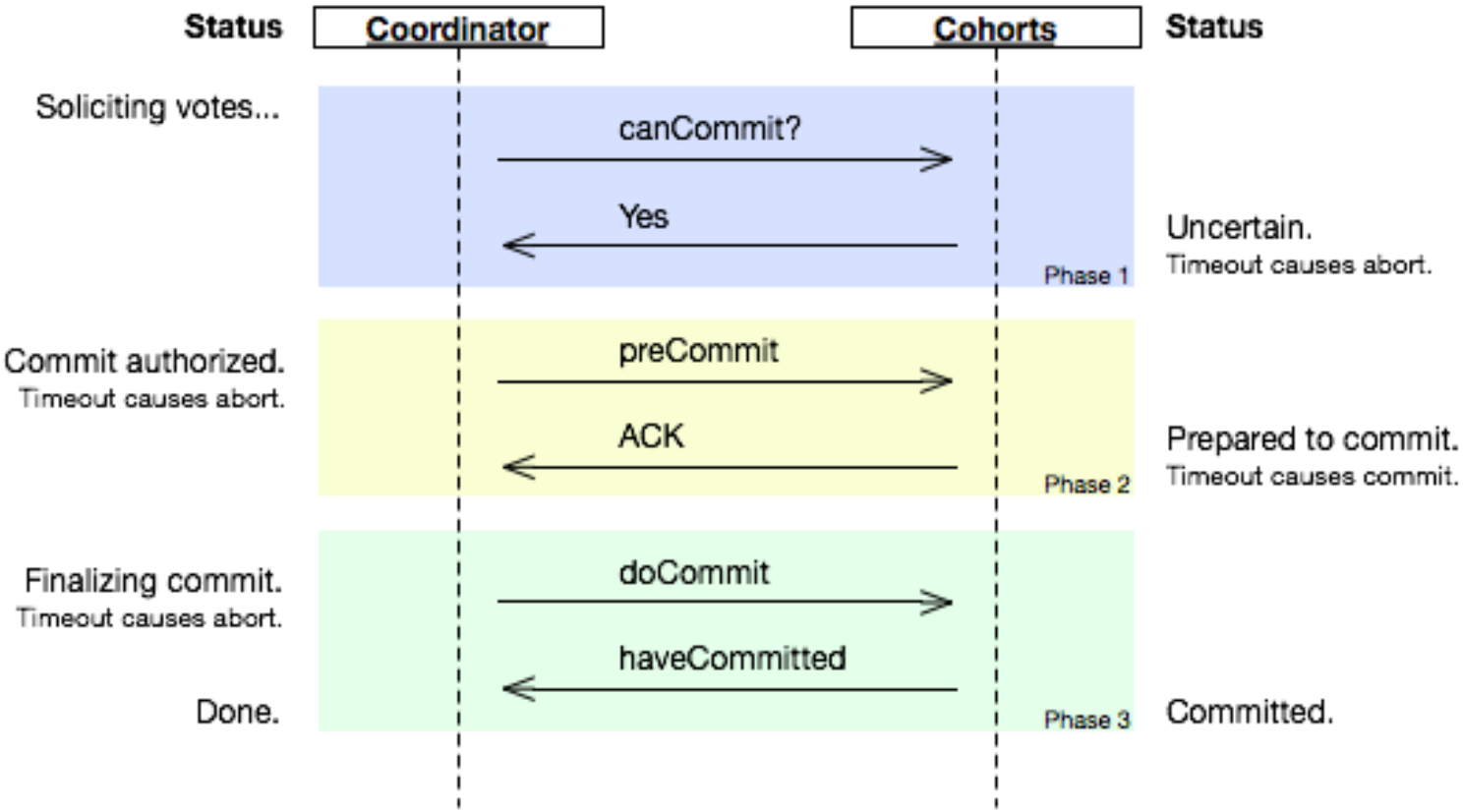
- 3PC should never block on node failures as 2PC did

Insight: 2PC suffers from allowing nodes to irreversibly commit an outcome before ensuring that the others know the outcome, too

Idea in 3PC: **split "commit/abort" phase into two phases**

- First communicate the outcome to everyone
- Let them commit only after everyone knows the outcome

3PC: Three Phase Commit Protocol (2)

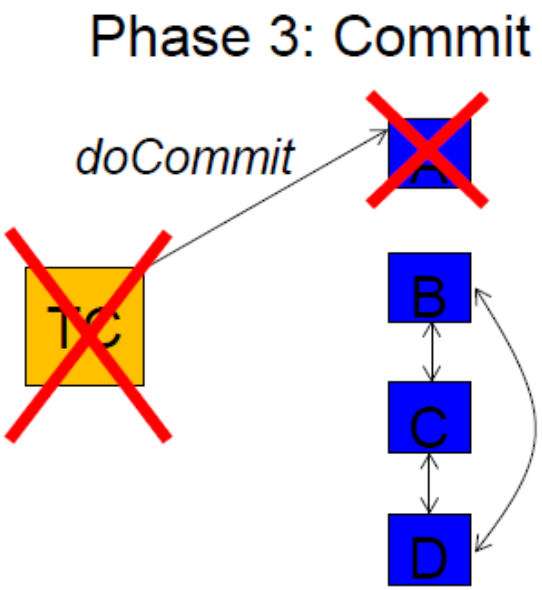


Can 3PC Solving the Blocking Problem? (1)

Assuming same scenario as before (TC, A crash), can B/C/D reach a safe decision when they time out?

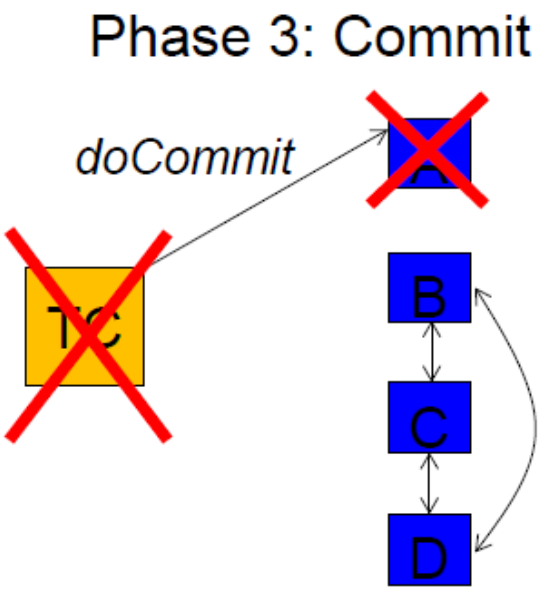
1. If one of them has received preCommit, …

2. If none of them has received preCommit, …



Can 3PC Solving the Blocking Problem? (2)

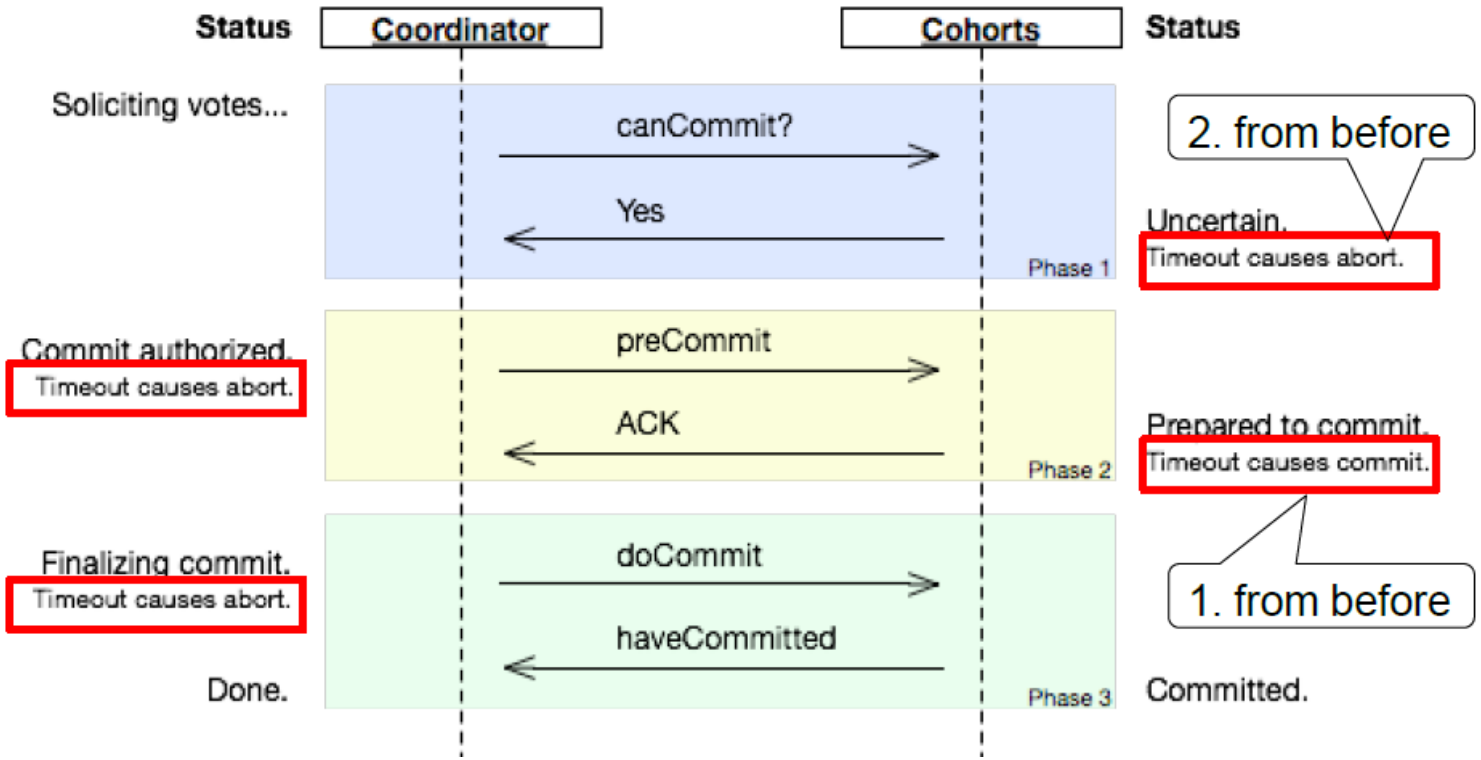
Assuming same scenario as before (TC, A crash), can B/C/D reach a safe decision when they time out?



- 1. If one of them has received preCommit, they can all commit
  - This is safe if we assume that A is DEAD and after coming back it runs a recovery protocol in which it requires input from B/C/D to complete an uncommitted transaction
  - This conclusion was impossible to reach for 2PC b/c A might have already committed and exposed outcome of transaction to world
- 2. If none of them has received preCommit, they can all abort
  - This is safe, b/c we know A couldn't have received a doCommit, so it couldn't have committed

3PC is safe for node crashes (including TC+participant)

3PC: Timeout Handling Specs (trouble begins)



But Does 3PC Achieve Consensus?

Liveness (availability): Yes

Doesn't block, it always makes progress by timing out

Safety (correctness): No

Can you think of scenarios in which original 3PC would result in inconsistent states between the replicas?

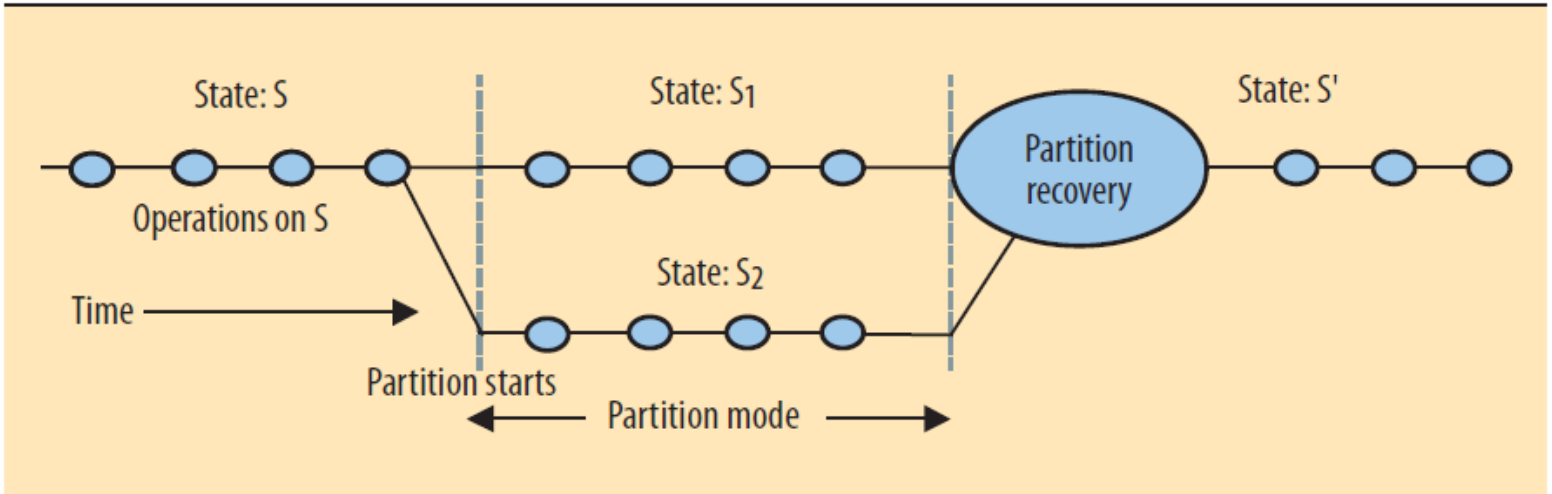
Two examples of unsafety in 3PC:

A hasn't crashed, it's just offline

TC hasn't crashed, it's just offline

} Network Partitions

Partition Management



Partition Detection

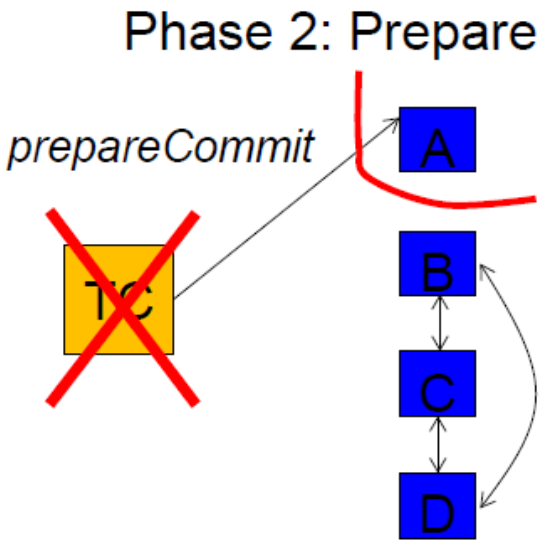
Activating Partition Mode

Partition Recovery

3PC with Network Partitions

One example scenario:

- A receives prepareCommit from TC
- Then, A gets partitioned from B/C/D and TC crashes
- None of B/C/D have received prepareCommit, hence they all abort upon timeout
- A is prepared to commit, hence, according to protocol, after it times out, it unilaterally decides to commit



Safety vs. liveness

So, 3PC is doomed for network partitions

The way to think about it is that this protocol's design trades safety for liveness

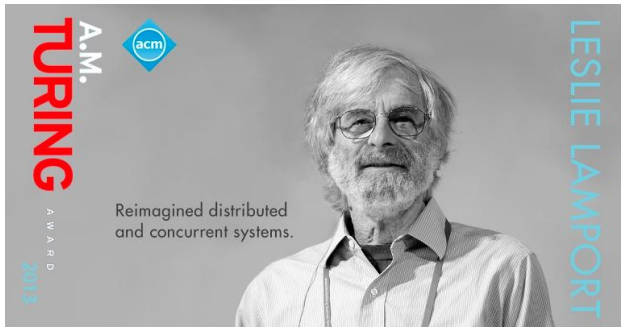
Remember that 2PC traded liveness for safety

Can we design a protocol that's both safe and live?

Paxos (1)

Leslie Lamport, 1989

Nearly synonymous with consensus



"The dirty little secret of the NSDI community is that at most five people really, truly understand every part of Paxos ;-)" – NSDI reviewer

"There are significant gaps between the description of the Paxos algorithm and the needs of a real-world system… the final system will be based on an unproved protocol." – Chubby authors

Paxos (2)

Related Papers:

1. Leslie Lamport, The Part-Time Parliament, *ACM Transactions on Computer Systems,* 16(2) :133-169, May 1998.

2. Leslie Lamport, Paxos Made Simple, Nov. 2001.

3. Tushar Chandra, Robert Griesemer, and Joshua Redstone, Paxos Made Live - An Engineering Perspective, June 2007.

Paxos (3)

The only known completely-safe and largely-live agreement protocol

Lets all nodes agree on the same value despite node failures, network failures, and delays

- Only blocks in exceptional circumstances that are vanishingly rare in practice

Extremely useful, e.g.:

- nodes agree that client X gets a lock
- nodes agree that Y is the primary
- nodes agree that Z should be the next operation to be executed

Paxos (4)

Widely used in both industry and academia

Examples:

Google: Chubby (Paxos-based distributed lock service)

- Most Google services use Chubby directly or indirectly

Yahoo: Zookeeper (Paxos-based distributed lock service) In Hadoop right now

MSR: Frangipani (Paxos-based distributed lock service)

UW: Scatter (Paxos-based consistent DHT)

Open source:

- libpaxos (Paxos-based atomic broadcast)
- Zookeeper is open-source and integrates with Hadoop

libpaxos - General purpose Paxos library

Settings | Report Duplicate

Apache ZooKeeper™

Paxos Properties

Safety

– If agreement is reached, everyone agrees on the same value

– The value agreed upon was proposed by some node

Fault tolerance (i.e., as-good-as-it-gets liveness)

– If less than half the nodes fail, the rest nodes reach agreement *eventually*

No guaranteed termination (i.e., imperfect liveness)

– Paxos may not always converge on a value, but only in very degenerate cases that are improbable in the real world

Lots of awesomeness

– Basic idea seems natural in retrospect, but why it works in any detail is incredibly complex!

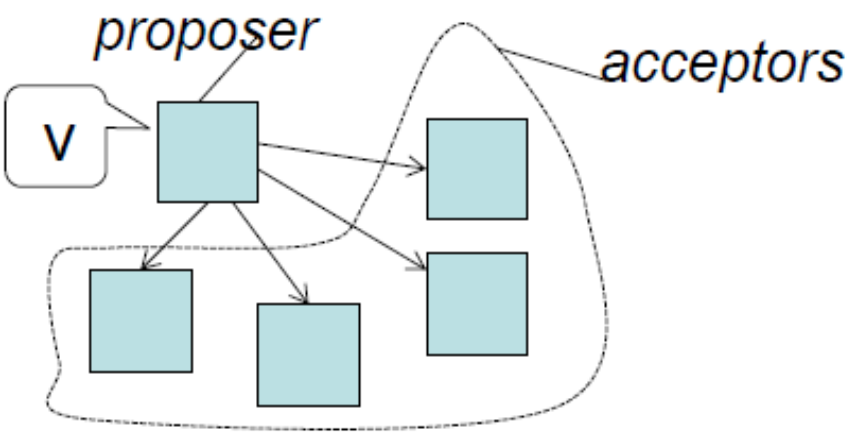
Basic Idea (1)

Paxos is similar to 2PC, but with some twists

One (or more) node decides to be coordinator (*proposer*)

Proposer proposes a value and solicits acceptance from others (*acceptors*)

Proposer announces the chosen value or tries again if it's failed to converge on a value



- Values to agree on:
- Whether to commit/abort a transaction
- Which client should get the next lock
- Which write we perform next
- What time to meet (party example)

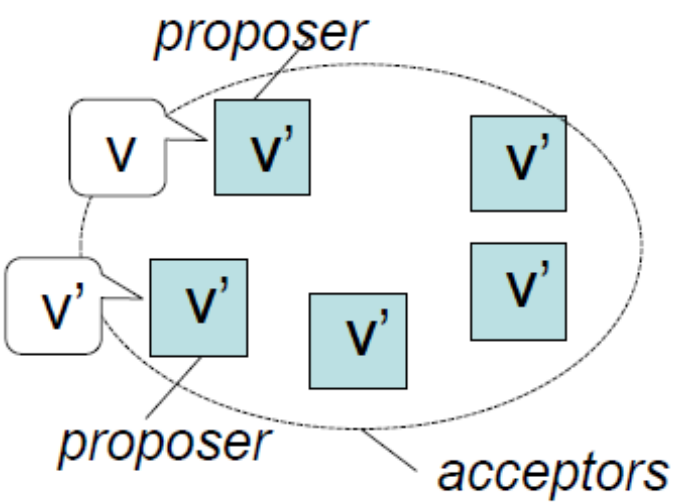
Basic Idea (2)

Paxos is similar to 2PC, but with some twists

One (or more) node decides to be coordinator (*proposer*)

Proposer proposes a value and solicits acceptance from others (*acceptors*)

Proposer announces the chosen value or tries again if it's failed to converge on a value



- Hence, **Paxos is egalitarian**: any node can propose/accept, no one has special powers
- Just like real world, e.g., group of friends organize a party – anyone can take the lead

Challenges

What if multiple nodes become proposers simultaneously?

What if the new proposer proposes different values than an already decided value?

What if there is a network partition?

What if a proposer crashes in the middle of solicitation?

What if a proposer crashes after deciding but before announcing results?

Core Differentiating Mechanisms

1. Proposal ordering

– Let nodes decide which of several concurrent proposals to accept and which to reject

2. Majority voting

– 2PC needs all nodes to vote Yes before committing

As a result, 2PC may block when a single node fails

– Paxos requires only a majority of the acceptors (half+1) to accept a proposal

As a result, in Paxos nearly half the nodes can fail to reply and the protocol continues to work correctly

Moreover, since no two majorities can exist simultaneously, network partitions do not cause problems (as they did for 3PC)

Implementation of Paxos

- Paxos has rounds; each round has a unique ballot id
- Rounds are asynchronous
  - Time synchronization not required
  - If you're in round *j* and hear a message from round *j+1*, abort everything and move over to round *j+1*
  - Use timeouts; may be pessimistic
- Each round itself broken into phases (which are also asynchronous)
  - Phase 1: A leader is elected (Election)
  - Phase 2: Leader proposes a value, processes ack (Bill)
  - Phase 3: Leader multicasts final value (Law)

Phase 1 – Election

Potential leader chooses a unique ballot id, higher than seen anything so far

Sends to all processes

Processes wait, respond once to highest ballot id

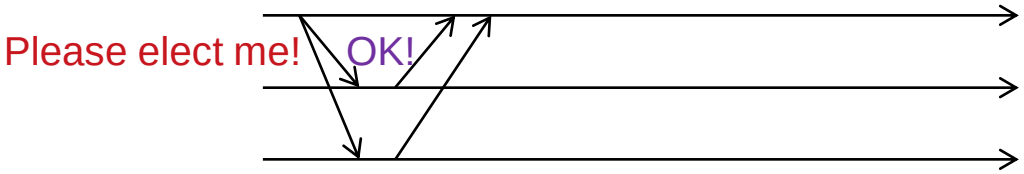
- If potential leader sees a higher ballot id, it can't be a leader
- Paxos tolerant to multiple leaders, but we'll only discuss 1 leader case
- Processes also log received ballot ID on disk

If a process has in a previous round decided on a value v', it includes value v' in its response

If majority (i.e., quorum) respond OK then you are the leader

- If no one has majority, start new round

A round cannot have two leaders (why?)

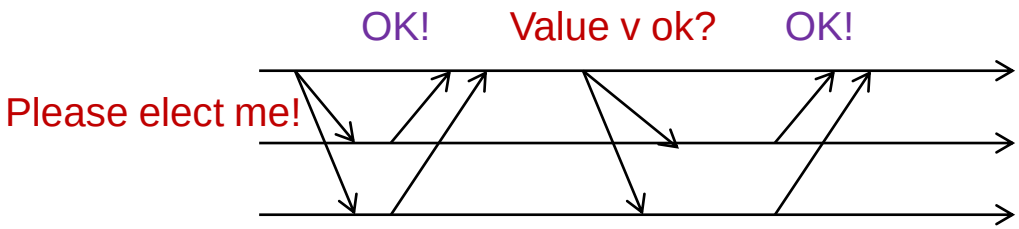


Phase 2 – Proposal (Bill)

Leader sends proposed value v to all

use v=v’ if some process already decided in a previous round and sent you its decided value v’

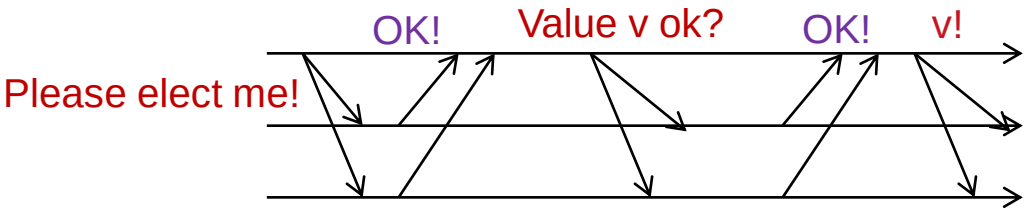
Recipient logs on disk; responds OK



Phase 3 – Decision (Law)

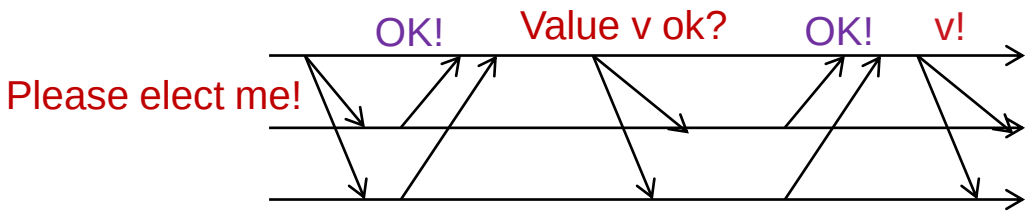
If leader hears a majority of OKs, it lets everyone know of the decision

Recipients receive decision, log it on disk



Which is the point of no-return? (1)

That is, when is consensus reached in the system



Which is the point of no-return? (2)

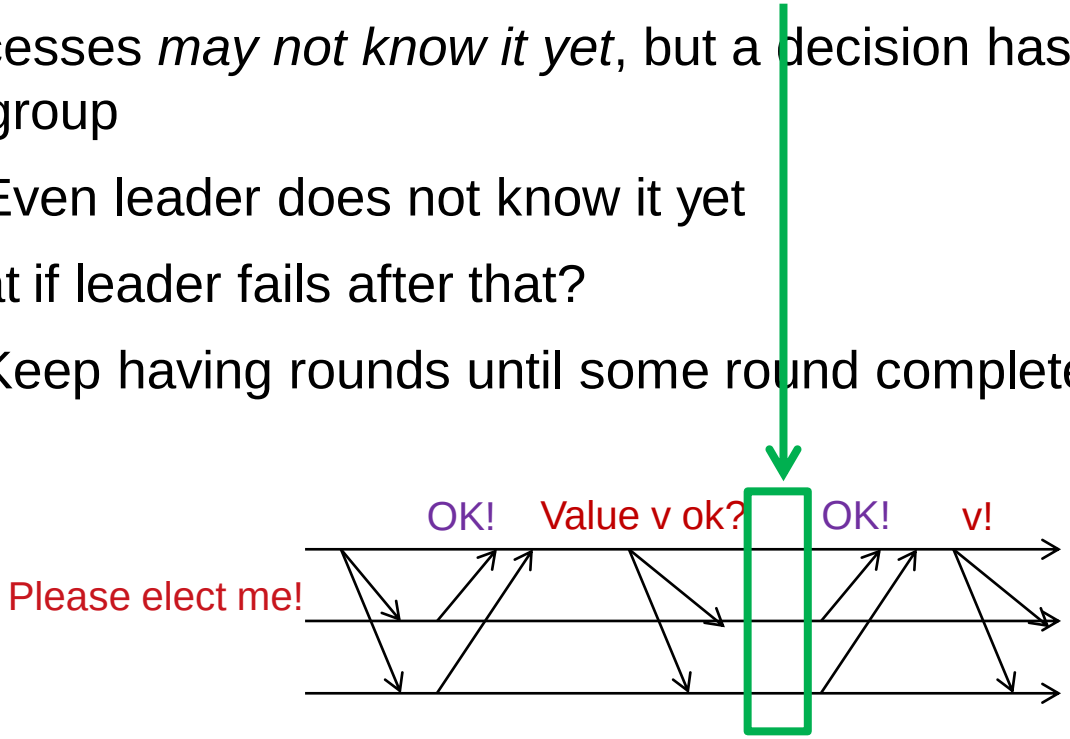
If/when a majority of processes hear proposed value and accept it (i.e., are about to/have respond(ed) with an OK!)

Processes *may not know it yet*, but a decision has been made for the group

- Even leader does not know it yet

What if leader fails after that?

- Keep having rounds until some round completes



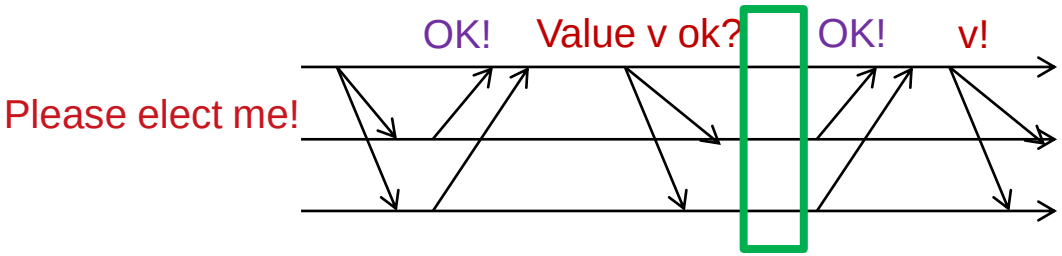
Safety

If some round has a majority (i.e., quorum) hearing proposed value v' and accepting it (middle of Phase 2), then subsequently at each round either: 1) the round chooses v' as decision or 2) the round fails

Proof:

- Potential leader waits for majority of OKs in Phase 1
- At least one will contain v' (because two majorities or quorums always intersect)
- It will choose to send out v' in Phase 2

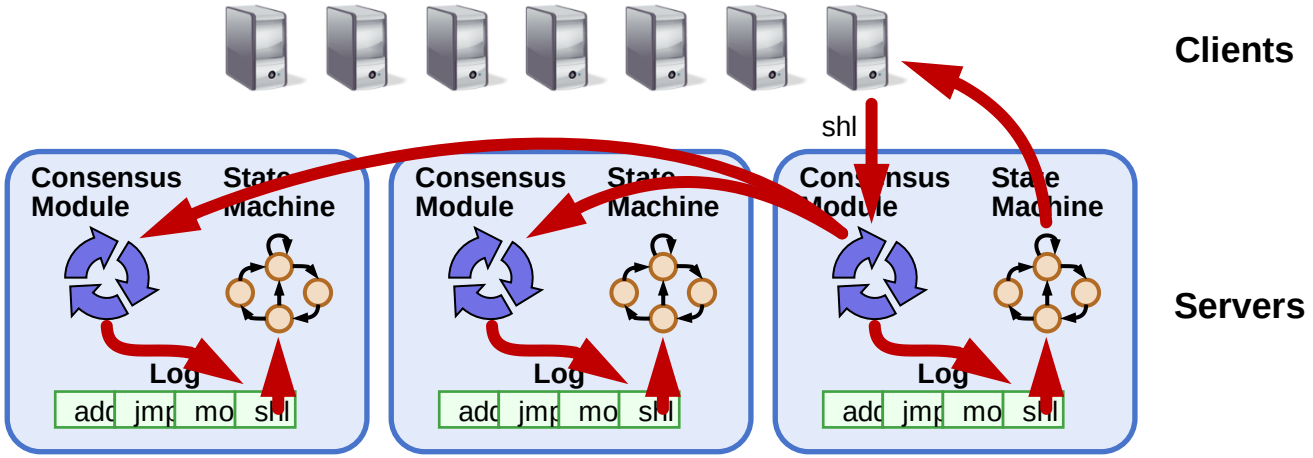
Success requires a majority, and any two majority sets intersect



What could go wrong?

- Process fails
  - Majority does not include it
  - When process restarts, it uses log to retrieve a past decision (if any) and past-seen ballot ids. Tries to know of past decisions.
- Leader fails
  - Start another round
- Messages dropped
  - If too flaky, just start another round
- Note that anyone can start a round any time
- Protocol may never end – tough luck, buddy!
  - Impossibility result not violated
  - If things go well sometime in the future, consensus reached

Classic Scenario in Cloud – Replicated Logs



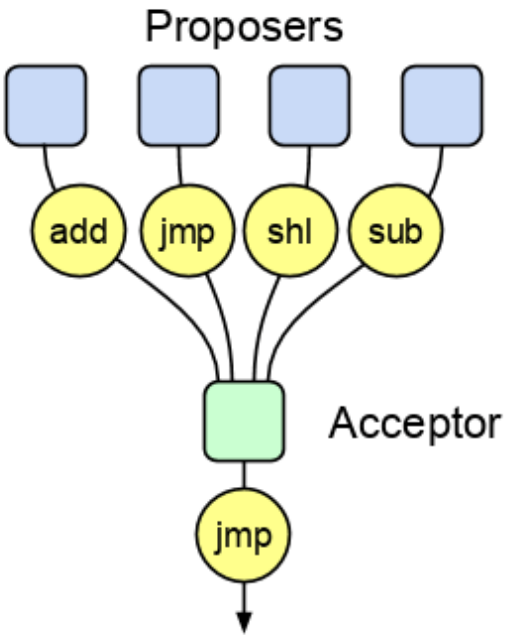
Replicated log => replicated state machine

All servers execute same commands in same order

Consensus module ensures proper log replication

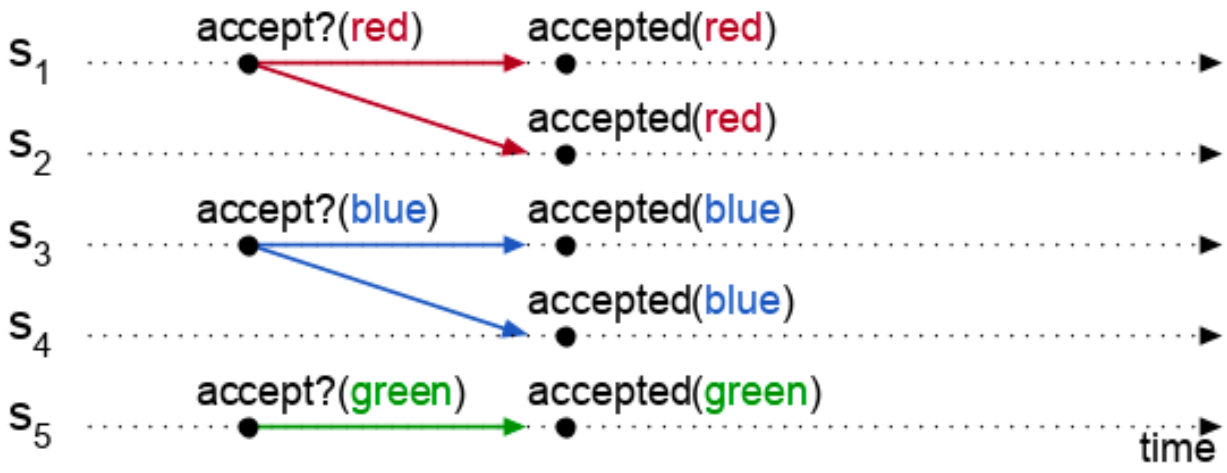
Problem for Paxos (1)

Problem 1: single acceptor → service unavailable



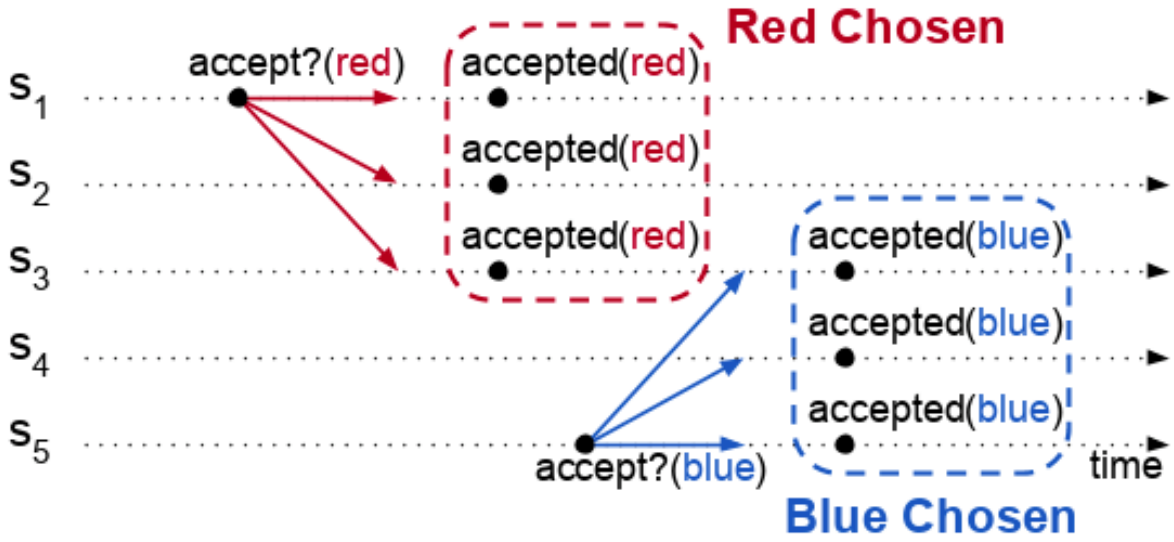
Problem for Paxos (2)

Problem 2: two acceptors have the same number of votes → no accepted results



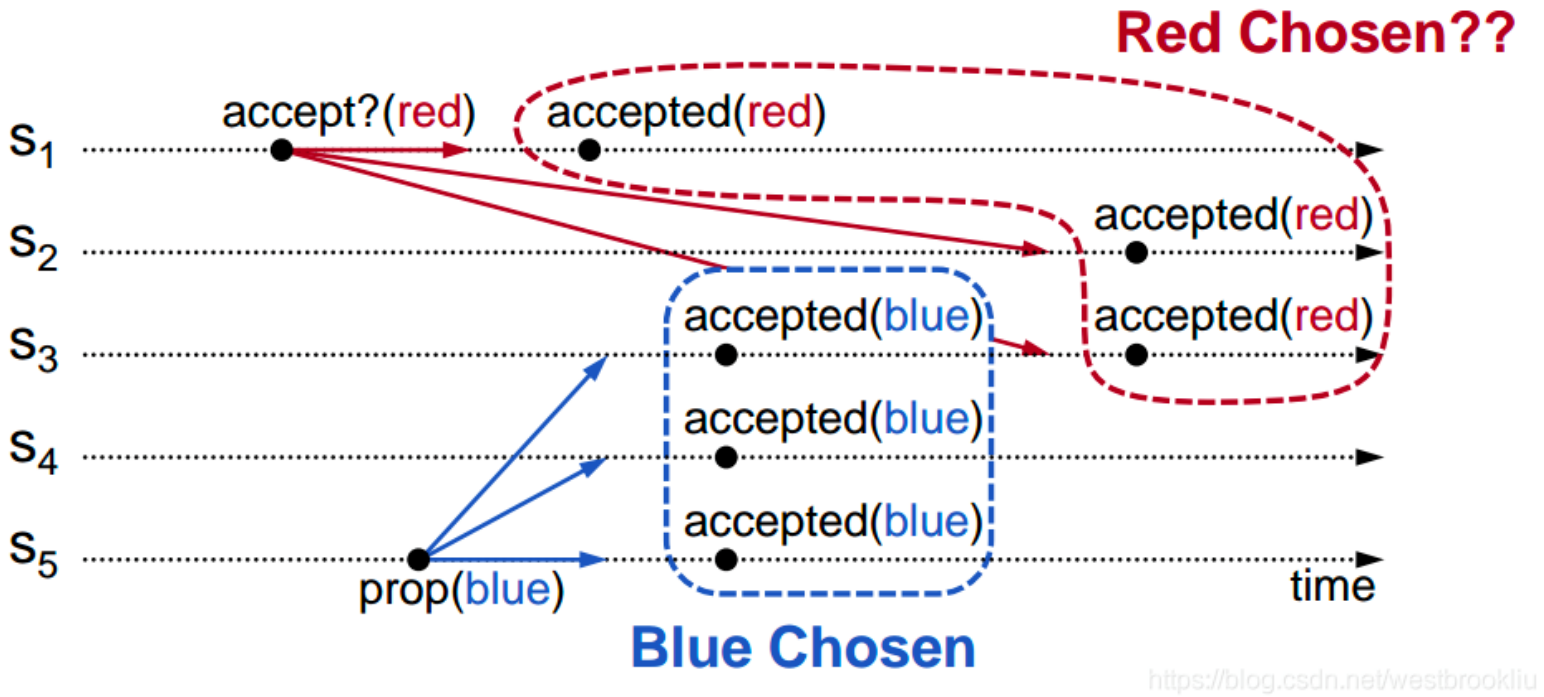
Problem for Paxos (3)

Problem 3a: selection conflicts (S3 has to accept two different values)



Problem for Paxos (4)

Problem 3b: another selection conflicts (S3 has to accept two different values)



https://blog.csdn.net/westbrookliu

Multi-Paxos

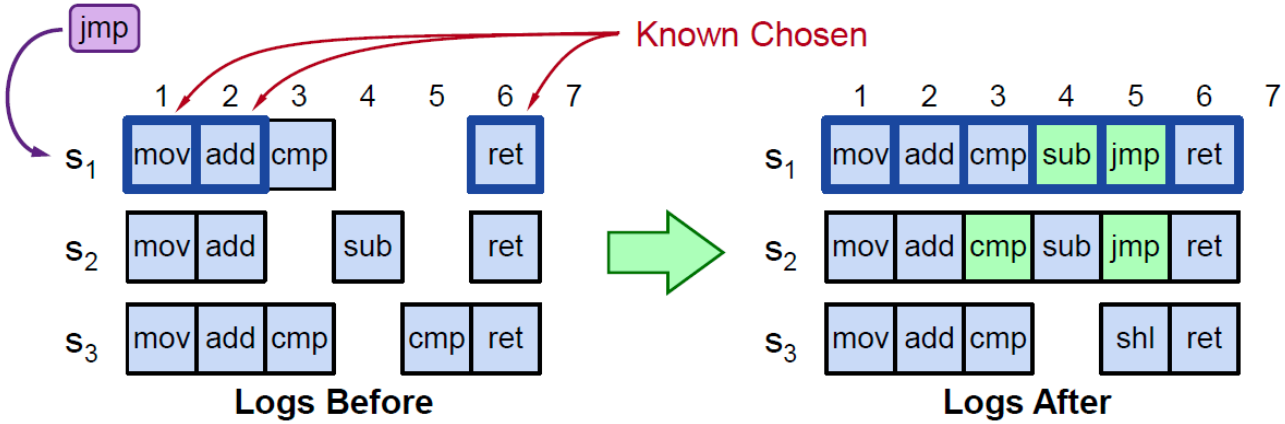
Multi-Paxos: Add an index for the replicated logs

Step 1: Find the first log record without chosen

Step 2: Run basic Paxos to propose the index with the proposal value requested by the client

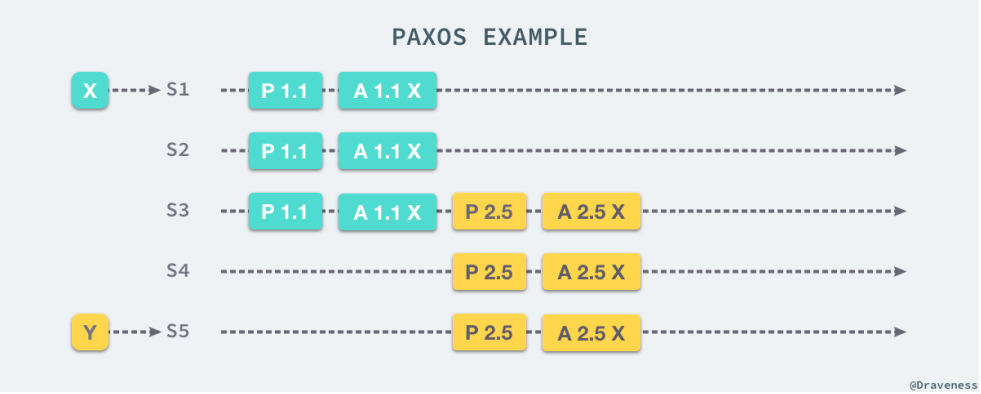
Step 3: Does prepare return accepted value?

- Yes: run this round of Paxos with accepted value, then go back to step 1
- No: chosen client value



Paxos vs. Multi-Paxos

Multi-Paxos: Reduce the RPCs and improve the efficiency



Multi-Paxos vs. Raft

Raft: a special version of multi-paxos. It limits the leader selection, which should have the all replicas of logs.

Raft

Diego Ongaro and John Ousterhout. In search of an understandable consensus algorithm. In *Proc. of USENIX Annual Technical Conference (USENIX ATC 14)*. Best Paper Award.

Website: https://raft.github.io/

Stars	Name	Primary Authors	Language	License	Leader Election + Log Replication?	Persistence?	Membership Changes?	Log Compaction?
35,091★	etcd/raft	Blake Mizerany, Xiang Li, Yicheng Qin	Go	Apache-2.0	Yes	Yes	Yes	Yes
24,523★	RethinkDB		C++	Apache-2.0	Yes	Yes	Yes	Yes
4,243★	hazelcast-raft	Mehmet Dogan, Ensar Basri Kahveci	Java	Apache-2.0	Yes	Yes	Yes	Yes
3,524★	dragonboat	Lei Ni	Go	Apache-2.0	Yes	Yes	Yes	Yes
8,973★	TiKV	Jay, ngaut, siddontang, tiancaiamao	Rust	Apache-2.0	Yes	Yes	Yes	Yes
4,391★	hashicorp/raft	Armon Dadgar	Go	MPL-2.0	Yes	Yes	Yes	Yes
2,530★	braft	Zhangyi Chen, Yao Wang	C++	Apache-2.0	Yes	Yes	Yes	Yes

Server States

At any given time, each server is either:

- **Leader**: handles all client interactions, log replication
- **Follower**: completely passive
- **Candidate**: used to elect a new leader

Normal operation: 1 leader, N-1 followers

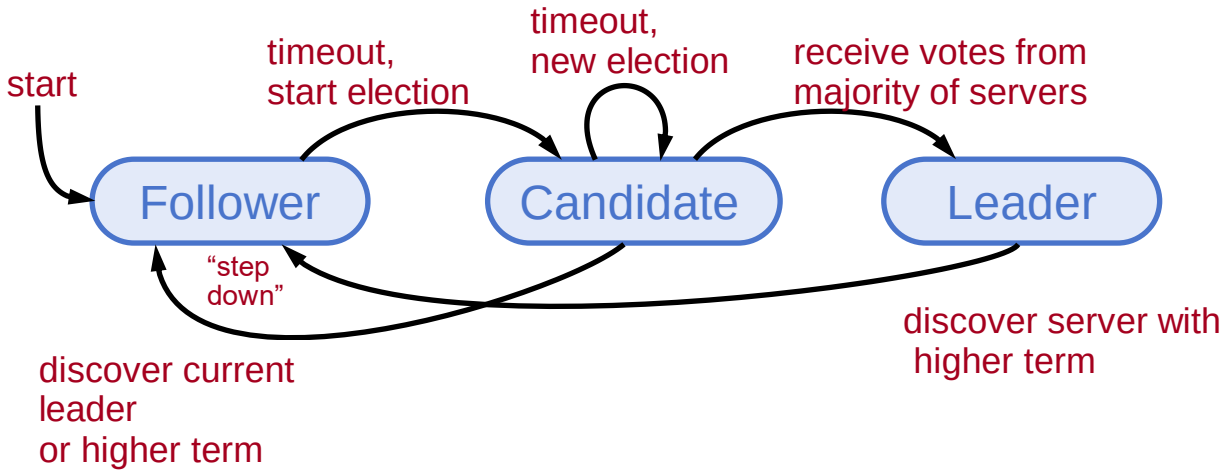
Follower Candidate Leader

Liveness Validation

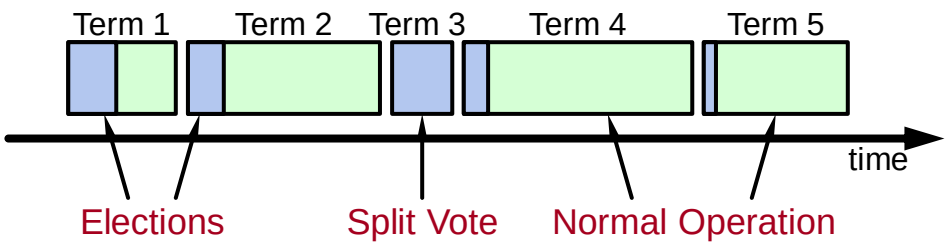
Servers start as followers

Leaders send heartbeats (empty AppendEntries RPCs) to maintain authority

If electionTimeout elapses with no RPCs (100-500ms), follower assumes leader has crashed and starts new election



Terms



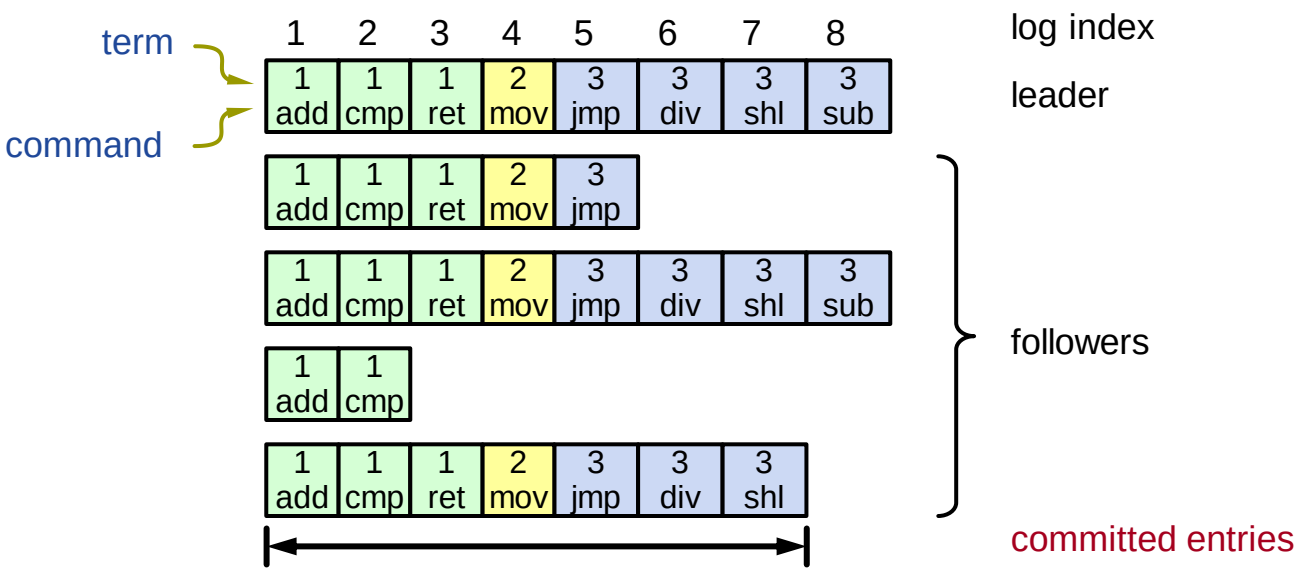
Time divided into terms

- Election (either failed or resulted in 1 leader)
- Normal operation under a single leader

Each server maintains current term value

Key role of terms: identify obsolete information

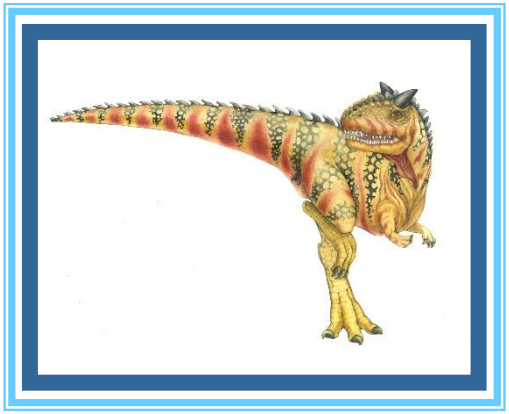
Log Structure



Log entry = < index, term, command >

Log stored on stable storage (disk); survives crashes

Entry committed if known to be stored on majority of servers

Durable / stable, will eventually be executed by state machines

End of Chapter 7